

東京情報大学大学院 総合情報学研究科

修士特別講義

- データサイエンスの基本と実践 -

- Kaggleの使い方とKaggleの分析コンペ
 - Kaggleの分析コンペで必要となる知識と技術の習得
-

東京情報大学 総合情報学部 総合情報学科

データサイエンス学系 生命・環境科学研究室

村上洋一 E-mail: ym202608@rsch.tuis.ac.jp



本講義のアウトライン

Kaggleに参加するため必要な準備をする

1. Kaggleとは
2. Kaggleで用いる機械学習 (KW: 教師あり学習)
3. Kaggleのアカウントの作成
4. Competitionsページの概要
5. 環境構築不要なNotebooksの使い方

最低限必要な機械学習テクニックを身につける

6. Kaggleへのsubmitの方法 (KW: Leaderboard, 特徴量エンジニアリング)
7. 一連の処理の流れ
8. 探索的データ分析 (KW: Pandas Profiling, 説明変数, 目的変数)
9. 仮説に基づく新しい特徴量の作成 (KW: 再現性)
10. いろいろな機械学習アルゴリズム (KW: sklearn, LightGBM, 勾配ブースティング)
11. ハイパーパラメータの調整 (KW: Optuna)
12. スコアの検証 (KW: ホールドアウト検証, Cross Validation)
13. アンアンブル学習 (KW: 多数決)

1. Kaggleとは

- **Kaggle** (カグル) : <https://www.kaggle.com/>
- 主に**機械学習モデル**を構築するコンペティション (コンペ) のプラットフォーム.
- 常に複数のコンペが開催され, 世界中の多くの参加者が企業や研究機関などが提供するデータから機械学習モデルを構築し, その性能を競い合っている.
- プログラミング言語の「Python」や「R」が実行できる「**Notebooks**」という環境も提供されている.
- ソースコードの公開や議論も活発で, 機械学習を学ぶのに適したプラットフォームになっている.

■ Kaggleのコンペの概要

- 企業などがデータと賞金を提供し, コンペの開催を依頼する.
- 参加者はデータを分析し予測結果を**submit** (提出) する.
- 予測結果は自動的に採点されスコアが表示される.
- コンペの期間中 (2~3ヶ月が多い) , 予測結果を何度もsubmitしてスコアを確認できる.
- コンペ終了後, スコアに基づく順位がつき, 上位者には賞金とメダルが付与される.
- メダルを一定数獲得すると, 上位の称号を獲得できる.
(称号) Grandmaster / Master / Expert / Contributor / Novice (初心者)
- アカウント作成時は「Novice」という称号

参考：メダルと称号の獲得条件 [2]

- メダルを獲得できる順位は、コンペの参加チーム数に応じて変動する。

	0~99チーム	100~249チーム	250~999チーム	1000~チーム
銅メダル	上位40%	上位40%	上位100チーム	上位10%
銀メダル	上位20%	上位20%	上位50チーム	上位5%
金メダル	上位10%	上位10チーム	上位10チーム+0.2%	上位10チーム+0.2%

- 「上位10チーム+0.2%」の「+0.2%」は、500チームごとに金メダルが与えられるチーム数が1増えるという意味である（ $500 \times 0.002 = 1$ ）
- 一定数のメダルを揃えると称号を獲得できる。獲得条件は以下のとおりである。

称号	獲得条件
Grandmaster	金メダル5個, うち1つはソロ（個人）での獲得
Master	金メダル1個, 銀メダル2個
Expert	銅メダル2個
Contributor	プロフィールの完成など ※
Novice	Kaggleの登録

- ※ 以下の条件を満たすことで、すぐにでもContributorの獲得が可能である。
- プロフィールへのbio（自己紹介）、居住地域、職業、所属組織の追加
 - アカウントのSMS認証
 - スクリプトの実行
 - コンペでのsubmit
 - NotebooksまたはDiscussionでのコメントとupvote（賛成票数）の実施

いいね

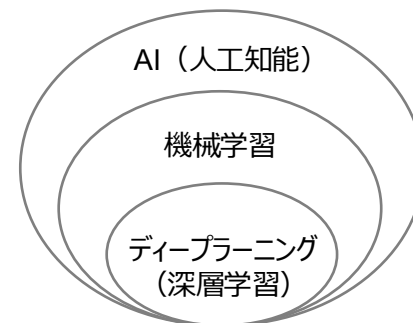
- 2024年1月時点、Competition Grandmaster 319名（うち日本人 62名） [3]
- Competitionの称号以外にも、Notebooks, Discussion, Datasetsの称号がある。

[2] Kaggle, "[Kaggle Progression System](#)"

[3] ResearchPort, "[Kaggle Grandmaster分析レポート 2024年 - update版 -](#)"

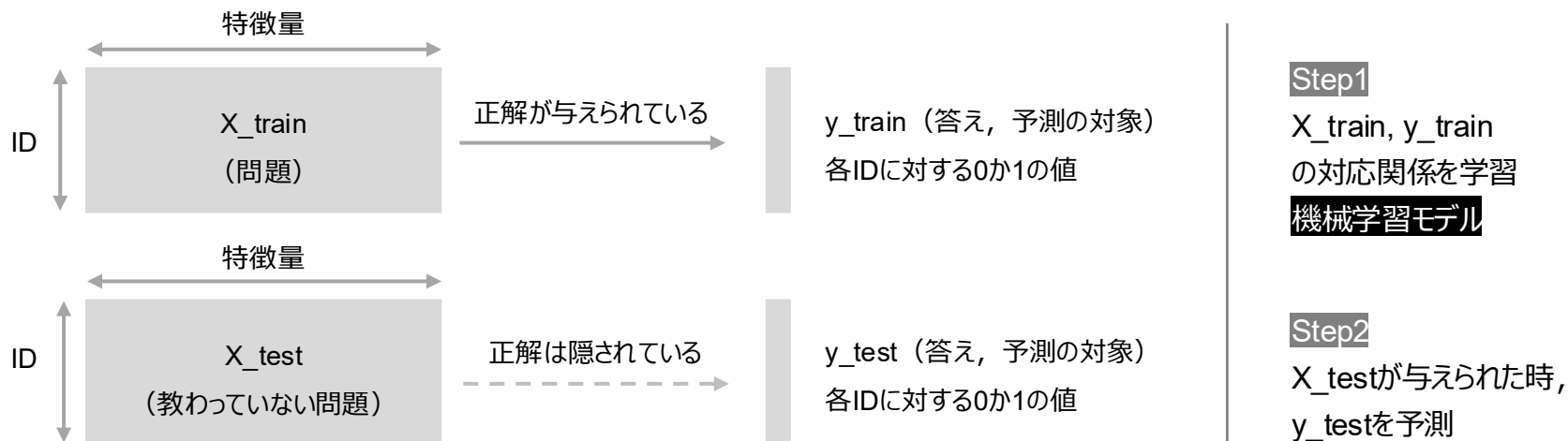
2. Kaggleで用いる機械学習 ①

- Kaggleで競うのは、機械学習モデルの性能である。
- 機械学習とは「コンピュータにヒトのような学習能力を獲得させるための技術の総称」である。
- 「人工知能（AI）」技術の中の一分野として位置付けられている。
- 機械学習は、学習方法によって大きく3つに分類できる。
 - 教師あり学習
 - 教師なし学習
 - 強化学習



■ 教師あり学習

- コンピュータに問題（X_train）と答え（y_train）の対をいくつか教えることで、教わっていない問題（X_test）にも正しく回答（y_test）できる汎化能力をコンピュータに獲得させる手法である。



2. Kaggleで用いる機械学習 ②

■ Kaggleで提供される情報

- 次のような情報が提供される.
 - 解くべき課題, 課題の評価方法
 - 学習用のデータセット (問題 X_{train} と 答え y_{train})
 - 性能を評価するデータセット (問題 X_{test} のみ)
- 参加者は**教師あり学習**を用いて, 可能な限り性能の高い予測を試みる.

例. 「[Titanic号の乗客ごとの生存の予測](#)」

- チュートリアル的な問題
- 課題の評価方法は「**正解率**」
- データセットとして「乗客に関する情報 (氏名・性別・チケット種別など)」が与えられる.

参考：教師なし学習と強化学習

教師なし学習

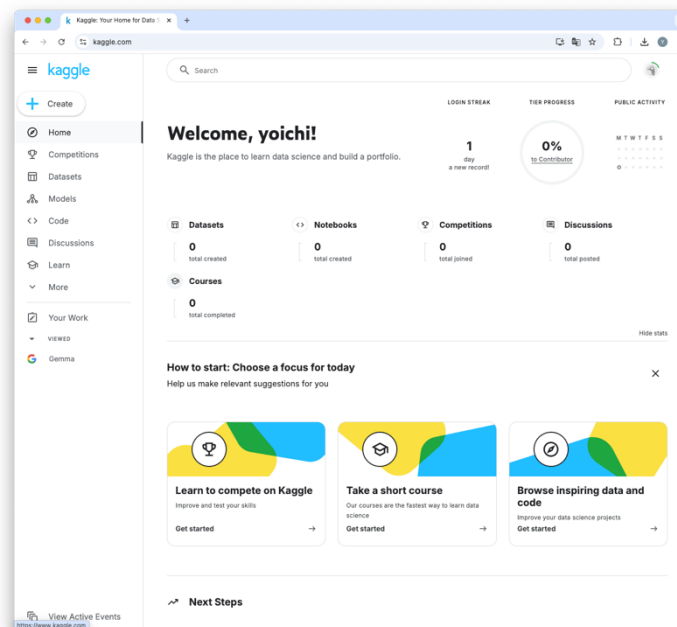
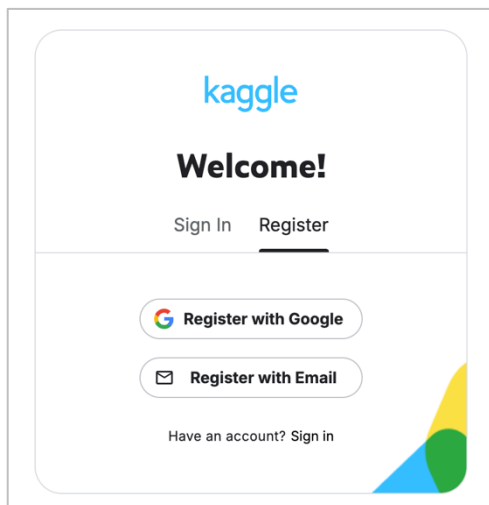
- 答えの分かっていないデータの集合から、有益な知識を獲得しようとする手法。
例. クラスタリング

強化学習

- 未知のデータに対応する能力をコンピュータに獲得させることが目的となる手法.
- 答えを直接教える代わりに、予測した答えの良さを評価し、評価が最も高くなるように学習を進めていく（予測した答えの良さを教えることで学習を進める）
- 例. ロボットの自動制御、コンピュータゲームなど
- 例. 囲碁
 - 「とある盤面で何を打つのが正解か」→ その後の展開の選択肢も多く難しい
 - 「となる手を打ったら勝った/負けた」という最終的な評価をもとに「どのような盤面ではどういう手を打つのが良い」という戦略を学ぶ.

3. Kaggleアカウントの作成

1. Kaggleのトップページ (<https://www.kaggle.com/>) にアクセスする.
2. 右上にある「Register」をクリックする.
3. アカウントの作成には, Googleのアカウントまたはメールアドレスが必要.
 - Googleのアカウントを用いる場合は「Register with Google」を選択する.
 - メールアドレスを用いる場合は「Register with Email」を選択する
 - アカウント作成時に入力する「Username」は, KaggleのプロフィールページのURLなどにも使われるID. あとから変更できないので, よく考えて登録する.



4. Competitionsページの概要 ①

- [開催中のコンペと終了したコンペが一覧表示](#)される（コンペのタイトルや参加チーム数など）。
- 例えば, 「[Titanic - Machin Learning from Disaster](#)」
- この「Titanic」は, [Kaggleに初めて取り組む人向けのコンペ](#)（Getting Started Prediction Competition）で, 賞金が設定されず, その代わりに**Knowledge**（知識）を得るためのコンペになっている。
- 他にSwag（記念品）やKudos（賞賛）を得るためのコンペもある。
- コンペに参加するためには, 通常, コンペトップページ右上の「**Join Competition**」をクリックする。
- クリック後, ルールの記載内容を確認し, 同意する場合は「I Understand and Accept」をクリックする。ルールに同意すると, 「Submit Prediction」のボタンが表示される。
- それぞれのコンペのタブには, 以下のコンテンツがある。

項目	コンテンツ
Overview	コンペの概要, 評価指標, メダル対象か否かなどの説明
Data	コンペのデータの説明
Notebooks	公開Notebookの一覧, Notebookの作成
Discussion	コンペの掲示板
Leaderboard	コンペの順位表
Rules	コンペのルール
Team	チーム名の変更やチームマージンリクエストの作成, 承認
My Submissions	自分のsubmit結果の一覧
Submit Predictions	自分のパソコンなどで作成したcsvファイルのsubmit

参考：コンペのルール，チーム

■ コンペのルール

- Titanicの場合，一日当たりのsubmit回数¹の上限が大きめに設定されている。
- 通常のコンペの場合，次のようなルールが一般的である。
 - 一人 1 アカウントで，複数アカウントからのsubmitを禁止する。
 - 「Private Sharing」は禁止されており，チーム外の者とソースコードなどを共有できない。
 - submit回数の上限が設定されている（2～5回までのコンペが多い）
 - チームの最大人数が設定されている（最近は5人までのコンペが多い）
 - チームマージをするときは，全員のsubmit数の合計が「1日の最大submit数 × コンペ開始からの日数」以内である必要がある。
 - コンペごとに，ルールは異なる。コンペに参加するときは必ずルールを確認してから同意すること。

■ チーム

- チーム名の設定や，チームメンバーの閲覧とチームリーダーの設定，チームメンバーの招待，招待中のリクエストの閲覧，招待されているリクエストの閲覧をすることができる。

参考 : Overview

- コンペの概要や評価指標などが表示される.
- コンペに参加する際に知っておいたほうが良い項目. 最初に確認すると良い.

■ Description

- コンペの目的や主催者の問題意識（何を解決したいのか）などが記載される.

■ Evaluation

- 評価指標とsubmitするcsvファイルの形式が記載される.
- submitするcsvファイルの形式はコンペごとに異なる.
- コンペによってはcsvではないファイル形式が指定される場合もある.

■ Timeline

- 次の期限などが表示される.

Entry deadline（ルールの同意期限）

- コンペに参加するにはこの日までにルールに同意する必要がある.

Team merger deadline（チームマージの期限）

- Final submission deadlineの一週間前に設定されることが多い.

Final submission deadline（提出の最終期限）

■ Prizes

- 入賞チーム数と、賞金額が記載される ※

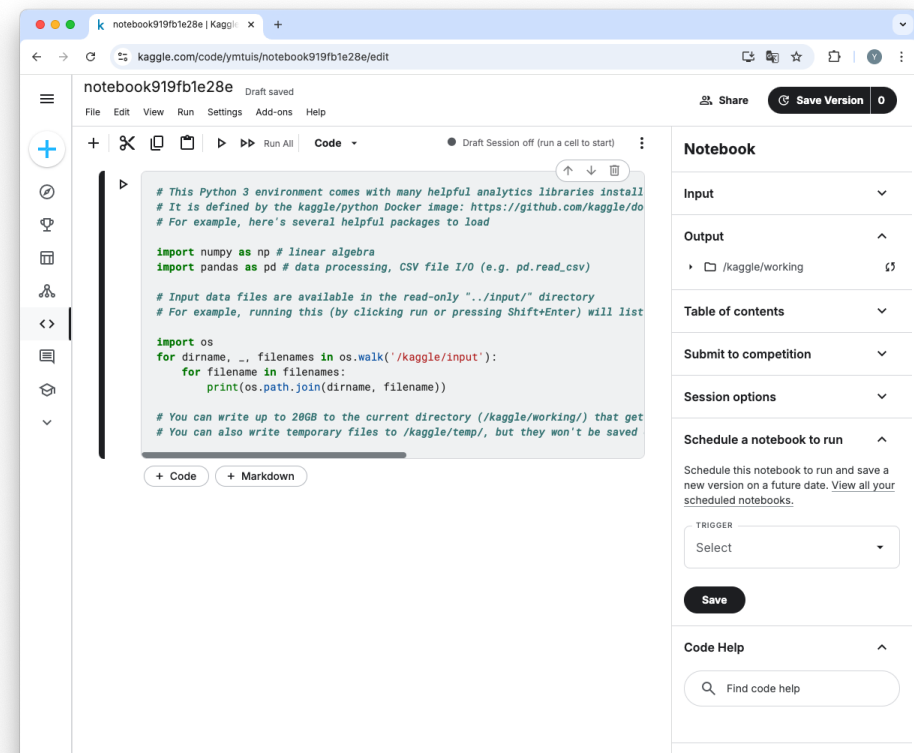
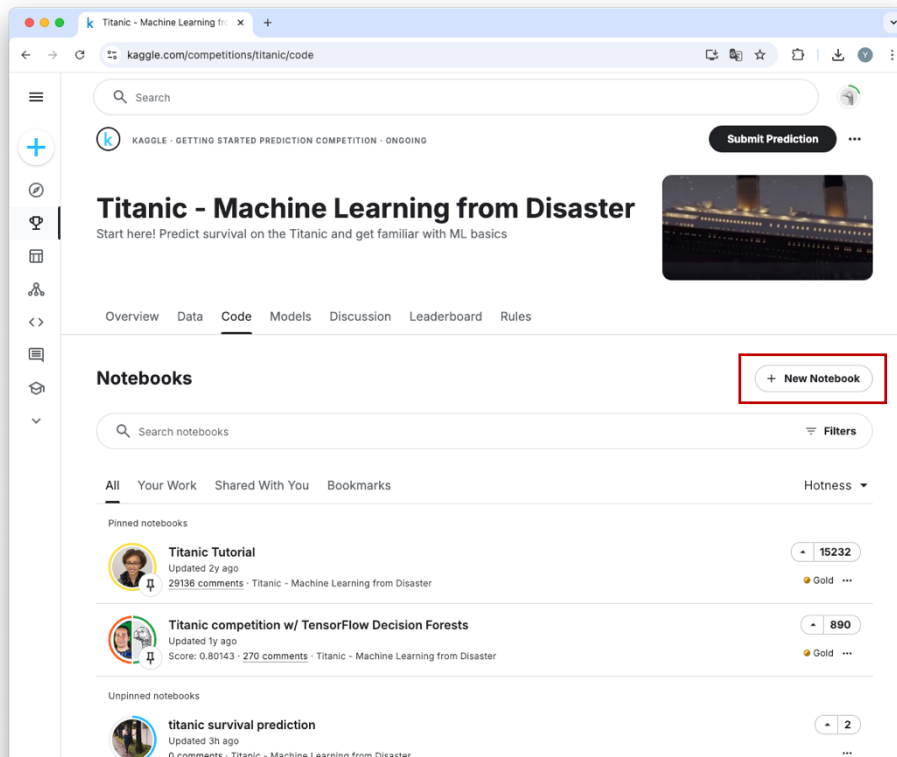
※ Titanicは、TimelineとPrizedはなく、TutorialsとFrequently Asked Questionsという項目がある.

5. 環境構築不要な「Notebooks」の使い方 ①

- PythonやRをブラウザ上で動かすことができる**Notebooks**環境が用意されている。
- 機械学習で用いられるさまざまなパッケージがあらかじめインストールされている。
- GPUも使用可能で、一般的なノートパソコン以上の性能が自由に使える環境が整ってる。

■ Notebookの作成

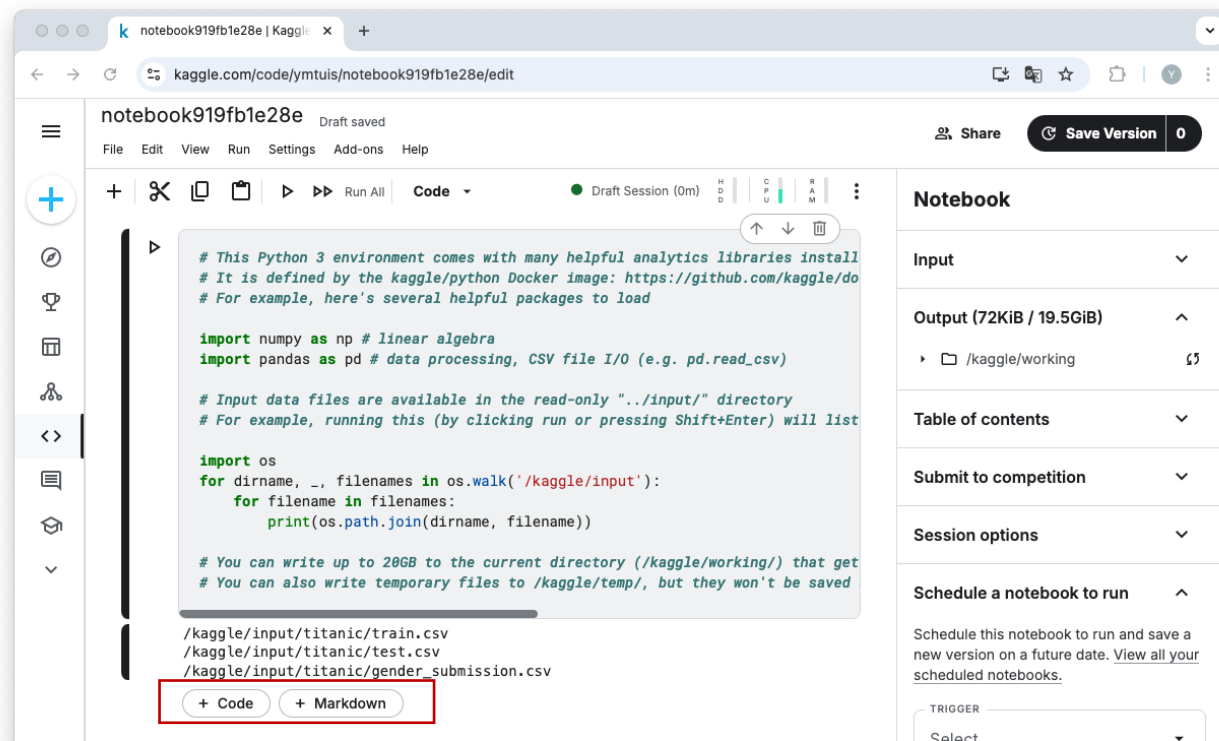
- 「Code」タブを選択した後、「+ New Notebook」をクリックする。



5. 環境構築不要な「Notebooks」の使い方 ②

■ ソースコードの実行

- Notebookは「セル」という単位でソースコードを記載し、セルごとにソースコードを実行できる。
- ソースコードを実行するには、セルの左側にある三角形をクリックする。または、セルをアクティブにしてショートカットキー「**Shiftキー+Enterキー**」を用いても実行できる。
- あらかじめ記載があるソースコードは、「/kaggle/input」にあるファイルを表示する。
- 実行すると、「**gender_submission.csv**」「**test.csv**」「**train.csv**」の3つのファイルが表示される。
- 新しくセルを作るには「+Code」または「+Markdown」をクリックする。



5. 環境構築不要な「Notebooks」の使い方 ③

■ ソースコードの実行

- 「+ Code」をクリックすると、ソースコードを記述するセルが追加される。
- 「+ Markdown」をクリックすると、説明文などをテキストで記載するセルが追加される。
- 「+ Code」をクリックして新しくセルを作成し、次のように「!pwd」と入力すると、現在の作業ディレクトリ「/kaggle/working」が表示される。
- Titanicの場合、以下のディレクトリ構造になっている。

```
kaggle
├── working
│   ├── input
│   │   └── titanic
│   │       ├── gender_submission.csv
│   │       ├── train.csv
│   │       └── test.csv
```

- 現在の作業ディレクトリから「kaggle/input/titanic/train.csv」を読み込むには、次のソースコードを実行する。

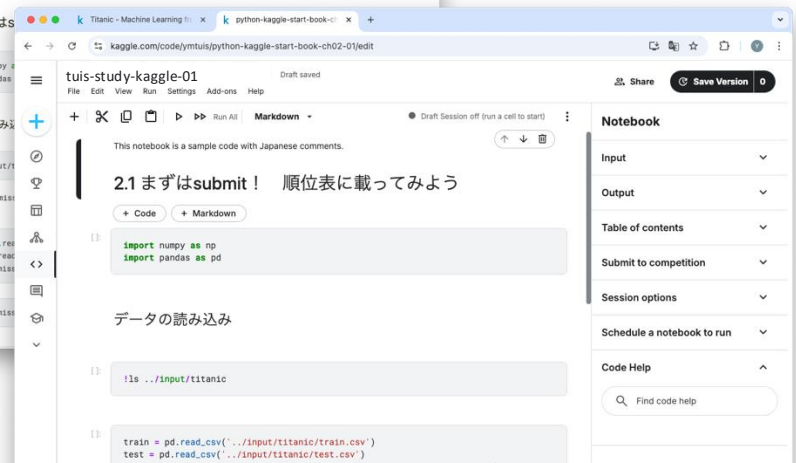
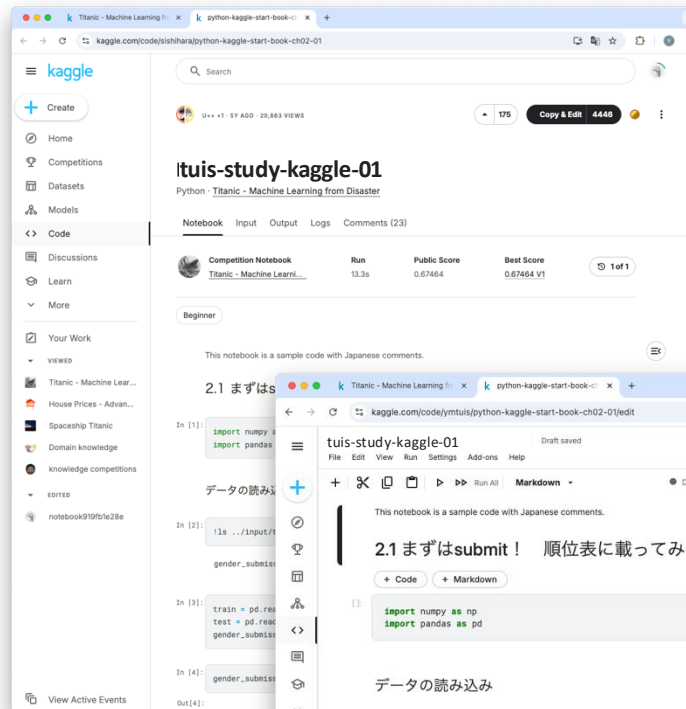
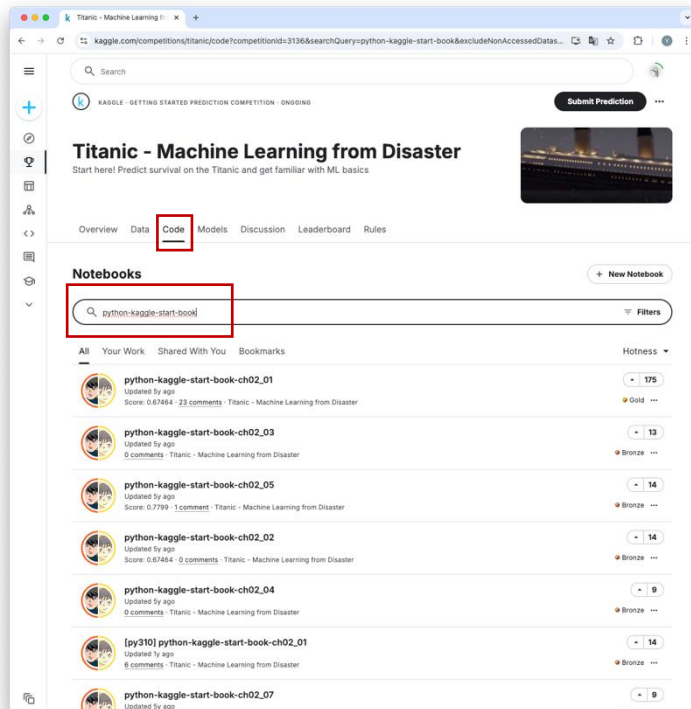
```
pd.read_csv('../input/titanic/train.csv')
```

- 「1+2」と入力して実行すると、計算結果「3」が表示される。
- 以上のように、Notebookはセルという単位でソースコードを実行することができ、結果を確認しながらソースコードを書いていくことができる。

5. 環境構築不要な「Notebooks」の使い方 ③

■ 公開されているNotebookの利用方法

- 他人が公開したNotebookを複製して、自分のNotebookとして利用可能である。
- 例えば、「Code」タブを開き、「Search notebooks」に「tuis-study-kaggle」と入力して検索すると今回のセミナーで使用するNotebookが表示される。
- 複製したいNotebookを選択し、右上の「Copy&Edit」をクリックすると、編集画面に遷移する。



参考：Notebookの使用状況と各種設定

- 左上の「File」には、Notebookの公開する項目「Share」や、Notebookで使用するプログラミング言語の選択する項目「Language」（PythonとRから選択可能）などがある。
- 公開設定には、自分だけが閲覧可能な「Private」、全体に公開する「Public」から選択可能。
- 「Setting」には、GPU・TPU使用の有無を選択する項目「Accelerator」がある。

The screenshot displays the Kaggle Notebook interface for a notebook titled 'notebook919fb1e28e'. The 'File' menu is highlighted with a red box, showing options like 'Share', 'Save Version', and '0'. The code editor shows a Python script for data processing. A 'Draft Session' pop-up window displays session details:

Draft Session	
No Accelerator	
Session	Disk
0m	2.2GiB
12 hours	Max 57.6GiB
CPU	
CPU	RAM
0.00%	389.8MiB
	Max 30GiB

The code editor contains the following Python code:

```
# This Python 3 environment comes with many helpful anaconda packages (see https://www.kaggle.com/competitions/titanic)
# It is defined by the kaggle/python Docker image: https://github.com/kaggle/docker-python
# For example, here's several helpful packages to load in this notebook:

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (https://pandas.pydata.org/pandas-docs/stable/10min.html)

# Input data files are available in the read-only "/kaggle/input" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list the files in the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/) that gets mounted as /kaggle/working.
# You can also write temporary files to /kaggle/temp/, but they won't be saved outside of the current session

/kaggle/input/titanic/train.csv
/kaggle/input/titanic/test.csv
/kaggle/input/titanic/gender_submission.csv
```

The right sidebar shows the 'Notebook' panel with sections for 'Input', 'Output (72KiB / 19.5GiB)', 'Table of contents', 'Submit to competition', 'Session options', and 'Schedule a notebook to run'.

本勉強会のアウトライン

Kaggleに参加するため必要な準備をする

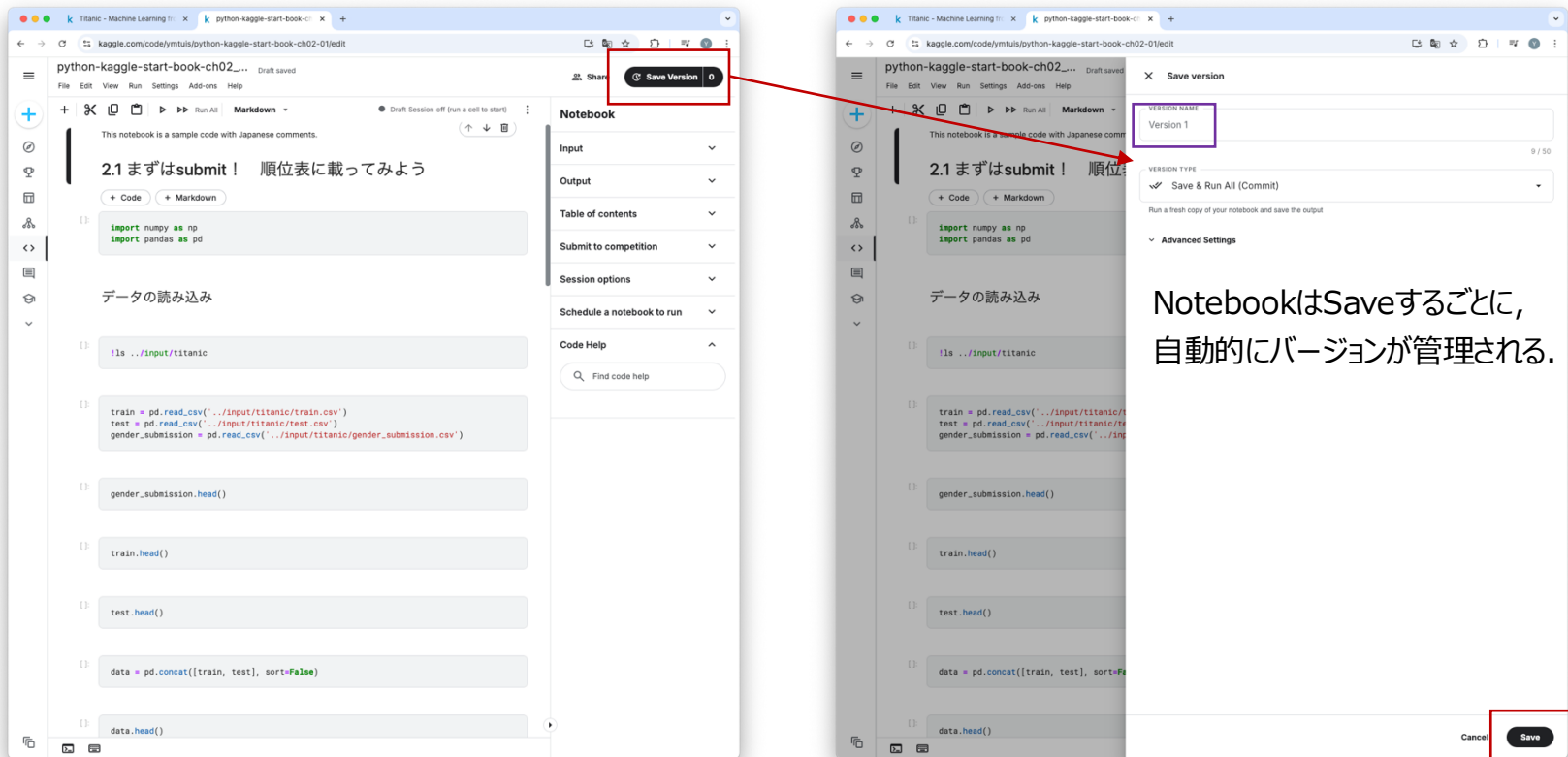
1. Kaggleとは
2. Kaggleで用いる機械学習 (KW: 教師あり学習)
3. Kaggleのアカウントの作成
4. Competitionsページの概要
5. 環境構築不要なNotebooksの使い方

最低限必要な機械学習テクニックを身につける

6. Kaggleへのsubmitの方法 (KW: Leaderboard, 特徴量エンジニアリング)
7. 一連の処理の流れ
8. 探索的データ分析 (KW: Pandas Profiling, 説明変数, 目的変数)
9. 仮説に基づく新しい特徴量の作成 (KW: 再現性)
10. いろいろな機械学習アルゴリズム (KW: sklearn, LightGBM, 勾配ブースティング)
11. ハイパーパラメータの調整 (KW: Optuna)
12. スコアの検証 (KW: ホールドアウト検証, Cross Validation)
13. アンアンブル学習 (KW: 多数決)

6. Kaggleへのsubmitの方法 ①

- 作成した機械学習モデルの予測結果は、次のいずれかの方法でsubmitできる。
 - KaggleのNotebook経由** / csvファイルを直接アップロード / Kaggle APIを利用
- Kaggleで公開している「**tuis-study-kaggle-01**」のサンプルコードを開き、「Copy & Edit」する。
- 右上の「Save Version」をクリックし、遷移したページで右下の「Save」をクリックする。



6. Kaggleへのsubmitの方法 ②

- Save後の画面の「Logs」タブを選択すると、実行結果のログを確認することができる。
- 「Output」タブを選択すると、このバージョンのNotebookでの予測結果が「submission.csv」というファイルで保存されていることが分かる。この画面で「Submit」をクリックする。

The first screenshot shows the Kaggle notebook interface with the 'Logs' tab selected. The notebook title is 'python-kaggle-start-book-ch02_01'. The 'Logs' tab displays a list of log messages, including 'Successfully ran in 10.4s'. The 'Output' tab is also visible, showing the 'submission.csv' file.

The second screenshot shows the 'Output' tab selected, displaying the 'submission.csv' file. The file is titled 'submission.csv (2.84 kB)'. The 'Submit' button is highlighted with a red box.

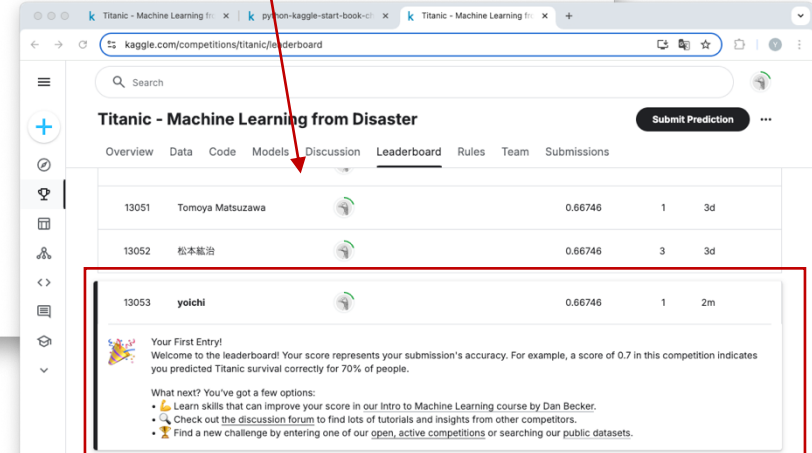
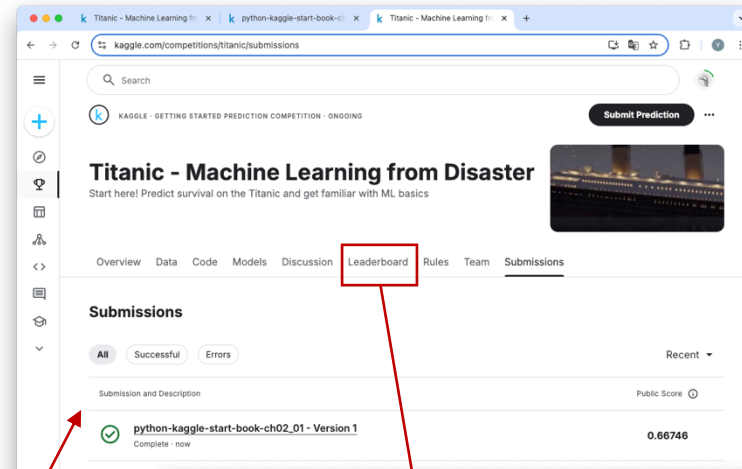
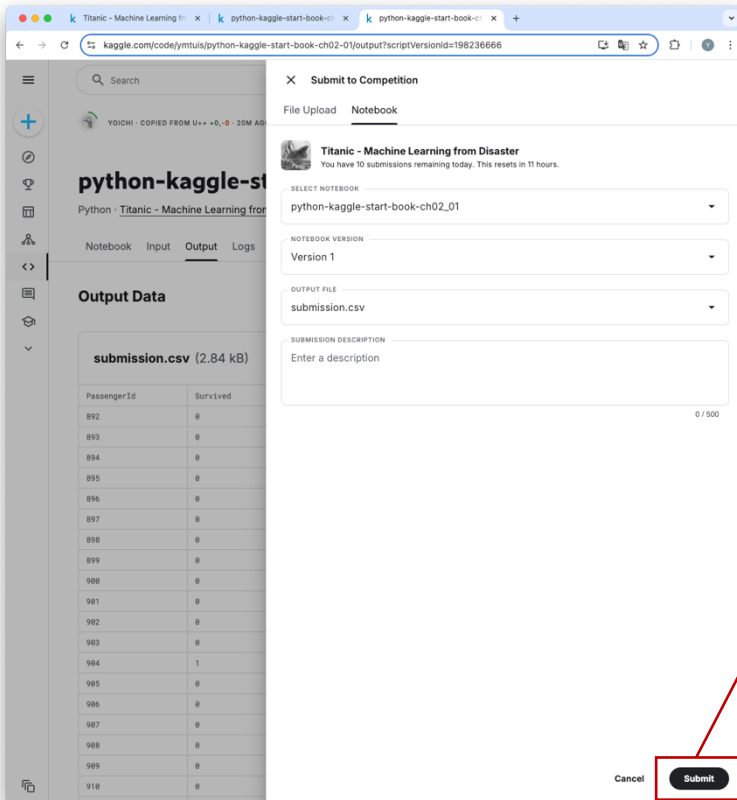
The third screenshot shows the 'Logs' tab selected, displaying the execution logs. The logs show the notebook being converted to HTML and the results being written to a file.

Time	#	Log Message
2.8s	1	[NbConvertApp] Converting notebook __notebook__.ipynb to notebook
4.2s	2	[NbConvertApp] Executing notebook with kernel: python3
8.8s	3	[NbConvertApp] Writing 29348 bytes to __notebook__.ipynb
9.6s	4	[NbConvertApp] Converting notebook __notebook__.ipynb to html
18.3s	5	[NbConvertApp] Writing 386245 bytes to __results__.html

PassengerId	Survived
892	0
893	0
894	0
895	0
896	0
897	0
898	0
899	0
900	0
901	0
902	0
903	0
904	1
905	0
906	0
907	0
908	0
909	0
910	0

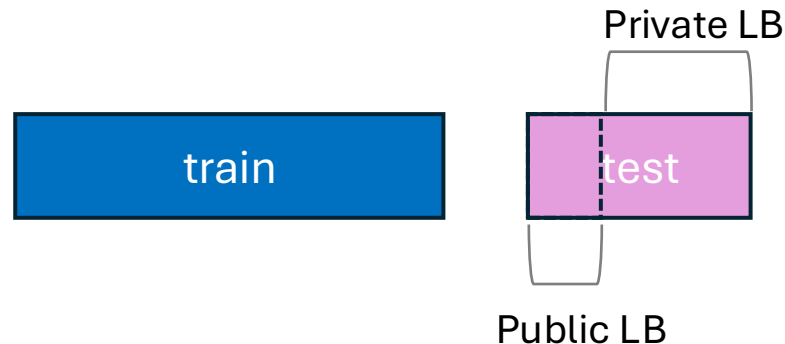
6. Kaggleへのsubmitの方法 ③

- Submit後、確認画面が表示され、右下の「Submit」をクリックすると、スコアが計算される
- スコアが計算され、今回の場合「0.66746」という値が算出される。
- 「Leaderboard」タブを選択すると、自分のアカウントが登場していることを確認できる。



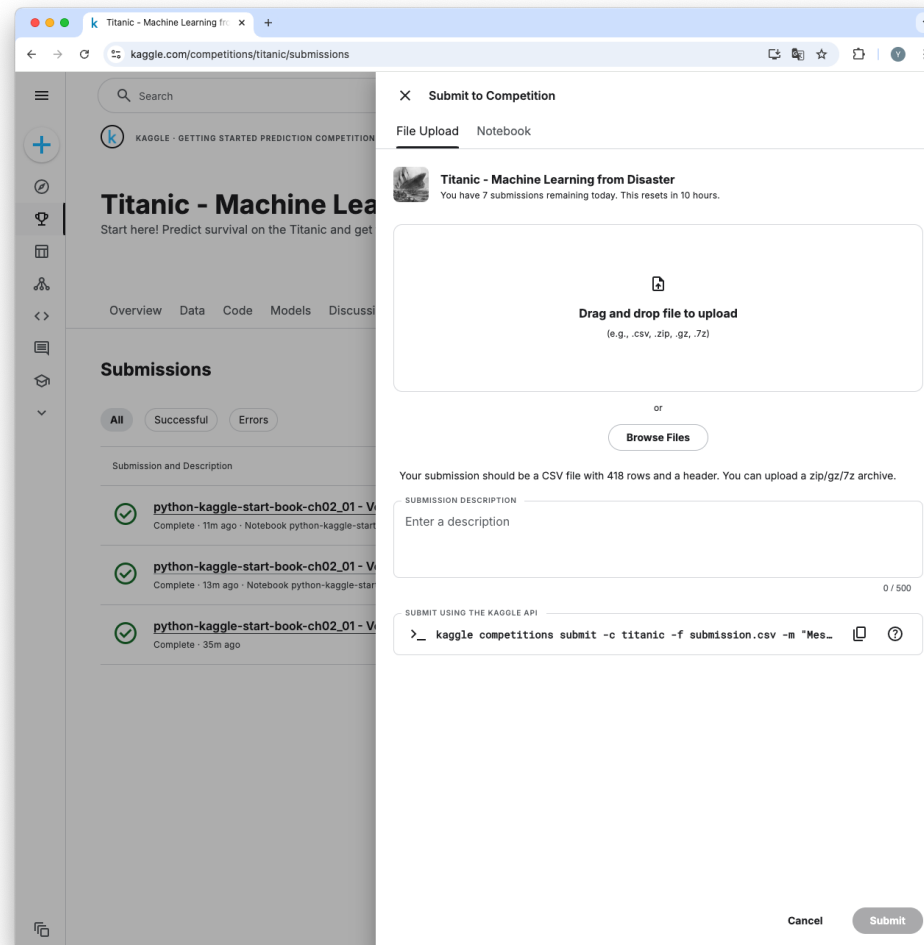
参考 : Leaderboard (LB)

- 順位は「Leaderboard (LB)」と呼ばれ, **Public LB** と **Private LB** の2つに分かれる.
- Public LB
 - **コンペ開催期間中から確認できるLB.**
 - テストデータの一部で計算したスコアのLBであり, 最終的な順位とは無関係である.
- Private LB
 - **最終順位となるLB.**
 - スコアは, Public LBのスコアに使われるテストデータとは別のテストデータで計算され, コンペ終了後にのみ確認できる.
- **Public LBのスコアを気にし過ぎず, Private LBで使われるテストデータのスコアが高いモデルをつくる.**
- Public LBとPrivate LBに使われるテストデータの割合は, コンペにより異なる.



参考：csvファイルを直接アップロードしたsubmit

- KaggleのNotebooks環境以外で作ったcsvファイルは、「Submit Predictions」タブから、そのファイルをアップロードすることでsubmitできる。



参考 : Kaggle APIを利用してsubmit

- Kaggle APIをインストールする。pipは、Pythonの標準的なパッケージ管理システム。

```
pip install kaggle
```

- 自分のAccountページ (<https://www.kaggle.com/<自分のユーザー名>/account>) に移動する。
- APIの箇所にある「Create New Token」をクリックし、「kaggle.json」をダウンロードする。
- このjsonファイルが、認証用のトークン。利用しているOSに応じて次のディレクトリに配置する。
 - Linux, macOS: 「~/kaggle/kaggle.json」
 - Windows: 「C:\Users\<Windows-username>\kaggle\kaggle.json」
- 自分だけが読み取りと書き込みができるように権限を設定する（初期設定完了）

```
chmod 600 ~/.kaggle/kaggle.json
```

- submitするコマンドは、各コンペの「Submissions」タブに記載がある。
- 例えば、Titanicの場合、次のコマンドになる。Messageの部分は自由に記入してよい。

```
kaggle competitions submit -c titanic -f submission.csv -m "Message"
```

- その他、Kaggle APIには次の機能などがある。
 - コンペ一覧の取得 / コンペのデータセットのダウンロード
 - Kaggle Datasetsのダウンロード・作成・更新 / Notebookのアップロード・ダウンロード

7. 一連の処理の流れ ①

Kaggle Notebook: [tuis-study-kaggle-01](#)

- 具体的な処理の流れは、以下のようにになっている。

1. パッケージの読み込み
2. データの読み込み
3. 特徴量エンジニアリング
4. 機械学習アルゴリズムの学習・予測
5. submit

■ パッケージの読み込み

- 以降の処理で使用する「パッケージ」をimportする。
- パッケージをimportすることで、標準では搭載されていない便利な機能を拡張して利用できる。

```
import numpy as np  
import pandas as pd
```

- NumPyは、数値計算に秀でたパッケージ。
- Pandasは、テーブル形式のデータ（データテーブル）を扱いやすいパッケージ。
- importはどこで実施しても問題ありませんが、冒頭でimportするのが望ましい。

7. 一連の処理の流れ ②

■ データの読み込み

- Kaggleから提供されたデータを読み込む。データの詳細は、Titanicのページの「[Data](#)」タブに記載。

```
train = pd.read_csv('../input/titanic/train.csv')
test = pd.read_csv('../input/titanic/test.csv')
gender_submission = pd.read_csv('../input/titanic/gender_submission.csv')
```

- 「**gender_submission.csv**」は、submit用のcsvファイルのサンプル.
 - 仮の予測として、女性のみが生存する（Survivedが1）という値が設定されている。
- 「**train.csv**」は、機械学習の学習用のデータ。
 - 乗客の性別・年齢などの属性情報、それぞれの乗客に対応する生存したか否かの情報（Survived）が格納されている。
- 「**test.csv**」は、予測を実施するデータ。
 - 乗客の性別・年齢などの属性情報のみが格納されており、学習用データの情報をもとに予測値を算出することになる。
 - train.csvと異なり、Survivedという列は存在しない。

```
gender_submission.head()
train.head()
test.head()
```

7. 一連の処理の流れ ③

■ データの読み込み（続き）

- 提供されているオリジナルデータは、例えば、NameやSexなどは文字列で格納されており、そのままでは機械学習アルゴリズムの入力にはできない。そのため機械学習アルゴリズムが扱える数値に形式に変換していく必要がある。
- 「Cabin」列には「NaN」（データの欠損）が78%ある。こうした欠損値は、一部の機械学習アルゴリズムではそのまま扱うことができますが、平均値や中央値など代表的な値で穴埋めする場合もある。

```
test.head()
```

	PassengerId	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	892	3	Kelly, Mr. James	male	34.5	0	0	330911	7.8292	NaN	Q
1	893	3	Wilkes, Mrs. James (Ellen Needs)	female	47.0	1	0	363272	7.0000	NaN	S
2	894	2	Myles, Mr. Thomas Francis	male	62.0	0	0	240276	9.6875	NaN	Q
3	895	3	Wirz, Mr. Albert	male	27.0	0	0	315154	8.6625	NaN	S
4	896	3	Hirvonen, Mrs. Alexander (Helga E Lindqvist)	female	22.0	1	1	3101298	12.2875	NaN	S

7. 一連の処理の流れ ③

■ 特徴エンジニアリング

- 次のような処理を「**特徴量エンジニアリング**」と呼ぶ
 - 読み込んだデータを機械学習アルゴリズムが扱える形に変換
 - 既存のデータから、機械学習アルゴリズムが予測する上で有用な新しい特徴量を作成

後者については、後ほど！

- 前者について、例えばSexの「male」と「female」をそれぞれ0と1に変換する.

```
data = pd.concat([train, test], sort=False)
data['Sex'].replace(['male', 'female'], [0, 1], inplace=True)
```

- 欠損値を穴埋めする処理も行う. 例えば, Fareについては, 平均値で補完する.

```
data['Fare'].fillna(np.mean(data['Fare']), inplace=True)
```

- オリジナルデータから特徴量エンジニアリングを経て, ようやく機械学習アルゴリズムに投入できる「X_train」「y_train」「X_test」というデータセットが得られる.

参考：特徴量の標準化 / 欠損値の補完

■ 特徴量の標準化

- Titanicの場合、例えば「Sex」は0か1のみの値しかとらないが、「Age」は最大で80の値も存在する。
- 各特徴量がとる値の範囲が異なると学習がうまく進まない場合がある、このような問題に対応するため、通常は各特徴量がとる値の範囲を調整 = 「標準化」する。
- 各特徴量の平均を0、標準偏差を1に変換する。
- `sklearn.preprocessing.StandardScaler()` が利用可能である。
- 近年、標準化などの特徴量の変換の影響を受けづらい機械学習アルゴリズムの利用率が高くなっている（「ランダムフォレスト」や「LightGBM」など）
- ここではLightGBMを機械学習アルゴリズムとして採用して進める。

■ 欠損値の補完

- LightGBMなど一部の機械学習アルゴリズムでは、欠損値をそのまま扱うことが可能である。
- 欠損値は、欠損していること自体に何らかの意味をもつ場合があり、やみくもに欠損値を補完してしまうのは得策では無い。
- 欠損値の扱いに関しては、次のような方法が考えられる。
 - 欠損値としてそのまま扱う
 - 代表的な値で欠損値を補完する
 - 他の特徴量から欠損値を予測して補完する
 - 欠損値か否かの情報を用いて新しい特徴量を作る

7. 一連の処理の流れ ④

■ 機械学習アルゴリズムの学習・予測

- 例えば、「ロジスティック回帰」という機械学習アルゴリズムを利用した場合、このアルゴリズムの振る舞いはハイパーパラメータという値で制御される。
- Logistic Regression()の括弧内の値が該当する（後ほど説明）

```
from sklearn.linear_model import LogisticRegression
clf = LogisticRegression(penalty='l2', solver='sag', random_state=0)
clf.fit(X_train, y_train)
```

- 学習を終えると、未知の特徴量（X_test）を与えて予測させることができる。
- y_predには0か1の予測値が格納されている。

```
y_pred = clf.predict(X_test)
```

■ submit

- Notebook経由でsubmitするために、予測値をcsvファイルとして保存する。

```
sub = pd.read_csv('../input/titanic/gender_submission.csv')
sub['Survived'] = list(map(int, y_pred))
sub.to_csv('submission.csv', index=False)
```

- Kaggleの運営側はy_testの中身を把握しているので、y_testと y_predが比較され、submitした予測値の性能がスコアとして返ってくる仕組みになっている。

8. 探索的データ分析 ①

Kaggle Notebook: [tuis-study-kaggle-02](#)

- スコアを向上するポイントの一つは、**特徴量エンジニアリング**。特に既存のデータから、新しい特徴量を作成する方法である。新しい特徴量の作成には、仮説と可視化を繰り返すサイクルが大切である
 - **仮説 ⇄ 可視化**
 - **予測性能に寄与しそうな仮説を立てる**
 - **可視化を実行する（予測性能に寄与する仮説を見つけるため、仮説を検証するため）**
- 個々人や問題によって、どちらが起点になるかが変わってくる
- ケース1) **ドメイン知識がある場合**
 - 「ドメイン知識」とは、取り組む業界や事業などについての知識を意味する
 - ドメイン知識をもっている分野の問題に取り組む場合、最初から仮説がいくつかある
 - 仮説を検証するような可視化を実施し、予測性能に寄与するかを確認する
 - 可視化の結果によって、改めて仮説を立てることになる
- ケース2) **ドメイン知識がない場合**
 - ドメイン知識がない場合、まずは仮説を立てるための**探索的なデータ分析**を実施することになる
 - いろいろな軸でデータを眺め、予測性能に寄与しそうな仮説を立てるのが目的になる
- 次の順で予測性能に寄与する新しい特徴量を見つけるための探索的な分析を実行する（探索的データ分析（Exploratory Data Analysis, EDA））
 1. **Pandas Profiling**で概要を確認する
 2. 各特徴量について目的変数（Survived）との関係を確認する

8. 探索的データ分析 ②

- 「Pandas Profiling」というパッケージをimportし、`pandas.DataFrame.profile_report()`とすることで、次の5項目からなるレポートが表示される。データの概要を分かりやすく表示してくれる。

1. Overview
2. Variables
3. Correlations
4. Missing Values
5. Sample

```
import pandas as pd
import pandas_profiling

train = pd.read_csv('../input/titanic/train.csv')
train.profile_report()
```

■ Overview

- Dataset infoには、データの行数（Number of observations）や列数（Number of variables）の型などが表示。Titanicのtrain.csvの場合、891行12列のデータ（891人分）であることが分かる。
- Missing cells は866で、866の欠損値があると分かる。
- Variable typesには、各列のデータ型の内訳が集計。Titanicの12列のデータ型は、次の通りである。
- Numeric (NUM): PassengerId, Age, SibSp, Parch, Fare
- Categorical (CAT): Pclass, Sex, Ticket, Cabin, Embarked, Name
- Boolean (BOOL): Survived
- Text: Name

Dataset info	
Number of variables	12
Number of observations	891
Missing cells	866 (8.1%)
Duplicate rows	0 (0.0%)
Total size in memory	322.0 KiB
Average record size in memory	370.1 B

Variables types	
CAT	6
NUM	5
BOOL	1

8. 探索的データ分析 ③

■ Overview (続き)

- Numeric と Categorical は対比されやすく、前者を「量的変数」、後者を「質的変数」と表現する場合もある。Boolean は真偽値を意味し、Titanicの場合、Survivedが当てはまる。Titanicでは 0 か 1 の数値で格納されている。
- 「Name」は、Categorical に分類されている。

■ Variables

- 各特徴量の概要が、データ型に応じたフォーマットで表示される
- Survived (目的変数)
 - 死亡(0)と生存(1)の人数。死亡(0)が549, 死亡(1)が342
- Age (乗船者の年齢)
 - Missing(欠損値, 年齢の情報がないデータ)が177
 - Mean(平均)は約26.7 / Minimum(最小値)は0.42 / Maximum(最大値)は80
 - ヒストグラムでは年齢の分布が分かる。20歳代から30歳代の乗客が多いと分かる。
- SibSp (同乗した兄弟姉妹と配偶者の人数)
 - 0人が68.2%, 1が23.5% ($=209/891=0.2345679\dots$)。2以上は少ない
- Parch (同乗した親と子どもの人数)
 - 0が約0.761%, 1が13.2% ($118/891=0.132435\dots$)

8. 探索的データ分析 ④

■ Variables (続き)

- Fare (運賃)
 - 小さい運賃が多いことが分かる.
- Pclass (チケットクラス)
 - 3枚のチケット. 1が(491, 24.2%), 2が(20.7%), 3が55.1%
- Sex (性別)
 - 男性(male)が64.9%, 女性(female)が35.2%
- Ticket (チケット番号)
 - 681種類のチケット番号.
 - 「1601」「347082」「CA2343」など, 最大で7の重複があることがわかる.
- Cabin (客室番号)
 - Distinct countから, 681種類のチケット番号があることがわかる
 - 「B96 B98」「G6」など, 最大で4の重複があることがわかる
- Embarked (乗船した港)
 - 欠損値2を除くと, Sが644 (72.3%), Cが168 (18.9%), Qが77 (8.6%)
- Name (乗客の名前)
 - Mr. や Mrs. などの継承が含まれている

参考：カテゴリ変数の特徴エンジニアリング

- 数値ではない文字列を入力として扱えない場合が多いため、カテゴリ変数を適当な数値に置き換える処理が必要になる。例えば、Sexの「male」と「female」をそれぞれ0と1に変換する。

```
data['Sex'].replace(['male', 'female'], [0, 1], inplace=True)
```

- Embarkedでは、Sを0, Cを1, Qを2に置換している。

```
data['Embarked'].fillna(('S'), inplace=True)  
data['Embarked'] = data['Embarked'].map({'S': 0, 'C': 1, 'Q': 2}).astype(int)
```

- この処理は、数値に変換したことで本来存在しない関係性が発展してしまうため、機械学習アルゴリズムにロジスティック回帰を用いる場合、適切ではない。
- 3種類以上の値をとるカテゴリ変数を数値に変換する際には、例えば、3列を展開し、それぞれの列で特定の値か否かを0もしくは1で表現する →「**One-Hotエンコーディング**」という。

Embarked		Embarked_S	Embarked_C	Embarked_Q
S	→	1	0	0
C		0	1	0
Q		0	0	1
C		0	1	0

- この変換ならば、不要な関係性をあたえることなくカテゴリ変数を処理可能である。

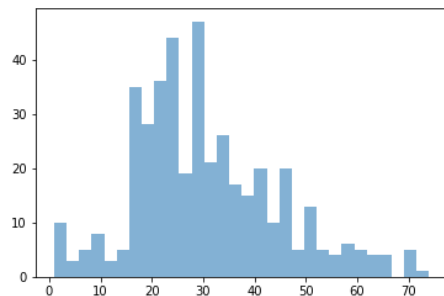
9. 仮説に基づく新しい特徴量の作成 ①

■ 各特徴量について目的変数との関係を確認する

- 各特徴量について、目的変数であるSurvived (死亡(0), 生存(1))との関係を確認し、予測性能に寄与する可能性のあるか調べる。
- 欠損値にある行は無視して、Survived=0 (死亡) 又はSurvived=1 (生存) である行の年齢を取得して、ヒストグラムを作成する（別々にプロット）

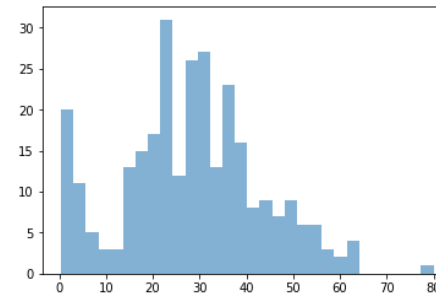
```
import matplotlib.pyplot as plt
plt.hist(train.loc[train['Survived']==0, 'Age'].dropna(), bins=30, alpha=0.5, label='0')
or plt.hist(train.loc[train['Survived']==1, 'Age'].dropna(), bins=30, alpha=0.5, label='1')
```

```
(array([10., 3., 5., 8., 3., 5., 35., 28., 36., 44., 19., 47., 21.,
        26., 17., 15., 20., 10., 20., 5., 13., 5., 4., 6., 5., 4.,
        4., 0., 5., 1.]),
array([ 1., 3.43333333, 5.86666667, 8.3, 10.73333333,
        13.16666667, 15.6, 18.03333333, 20.46666667, 22.9,
        25.33333333, 27.76666667, 30.2, 32.63333333, 35.06666667,
        37.5, 39.93333333, 42.36666667, 44.8, 47.23333333,
        49.66666667, 52.1, 54.53333333, 56.96666667, 59.4,
        61.83333333, 64.26666667, 66.7, 69.13333333, 71.56666667,
        74. ]),
<a list of 30 Patch objects>)
```



Survived=0 (死亡)

```
(array([20., 11., 5., 3., 3., 13., 15., 17., 31., 12., 26., 27., 13.,
        23., 16., 8., 9., 7., 9., 6., 6., 3., 2., 4., 0., 0.,
        0., 0., 0., 1.]),
array([ 0.42, 3.07266667, 5.72533333, 8.378, 11.03066667,
        13.68333333, 16.336, 18.98866667, 21.64133333, 24.294,
        26.94666667, 29.59933333, 32.252, 34.90466667, 37.55733333,
        40.21, 42.86266667, 45.51533333, 48.168, 50.82066667,
        53.47333333, 56.126, 58.77866667, 61.43133333, 64.084,
        66.73666667, 69.38933333, 72.042, 74.69466667, 77.34733333,
        80. ]),
<a list of 30 Patch objects>)
```



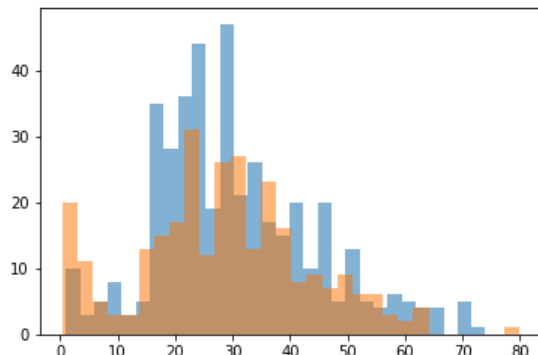
Survived=1 (生存)

9. 仮説に基づく新しい特徴量の作成 ②

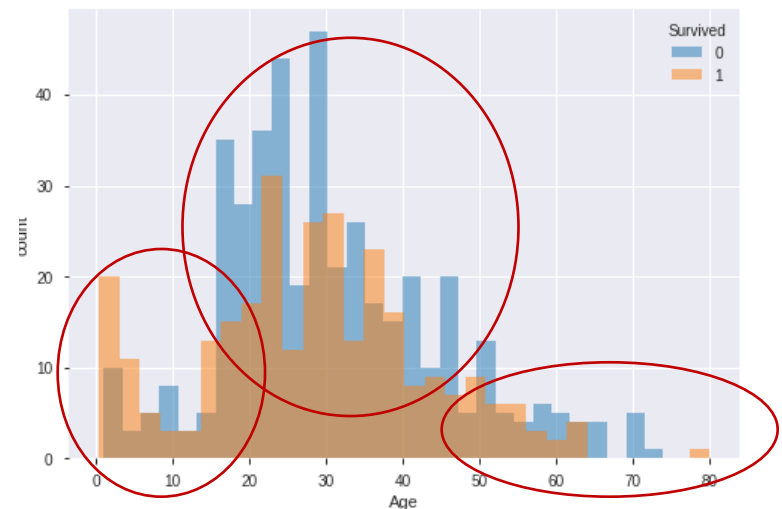
- 欠損値がある行は無視して, Survived=0 (死亡) である行の年齢を取得して作成したヒストグラムと, Survived=1 (生存) である行の年齢を取得して作成したヒストグラムを重ねてプロットする。
- 目的変数ごとに, 年齢ごとの生存者数 (生存率) が分かる。
- 若い人や年配の人は生存率が高く, 20歳代から30歳代の人は生存率が低くなっている

```
plt.hist(train.loc[train['Survived']==0, 'Age'].dropna(), bins=30, alpha=0.5, label='0')  
plt.hist(train.loc[train['Survived']==1, 'Age'].dropna(), bins=30, alpha=0.5, label='1')
```

```
(array([20., 11., 5., 3., 3., 13., 15., 17., 31., 12., 26., 27., 13.,  
       23., 16., 8., 9., 7., 9., 6., 6., 3., 2., 4., 0., 0.,  
       0., 0., 0., 1.]),  
 array([ 0.42,  3.07266667,  5.72533333,  8.378, 11.03066667,  
        13.68333333, 16.336, 18.98866667, 21.64133333, 24.294,  
        26.94666667, 29.59933333, 32.252, 34.90466667, 37.55733333,  
        40.21, 42.86266667, 45.51533333, 48.168, 50.82066667,  
        53.47333333, 56.126, 58.77866667, 61.43133333, 64.084,  
        66.73666667, 69.38933333, 72.042, 74.69466667, 77.34733333,  
        80.    ]),  
<a list of 30 Patch objects>)
```



凡例 (legend)を付けて,
2つのヒストグラムを重ねてプロットする

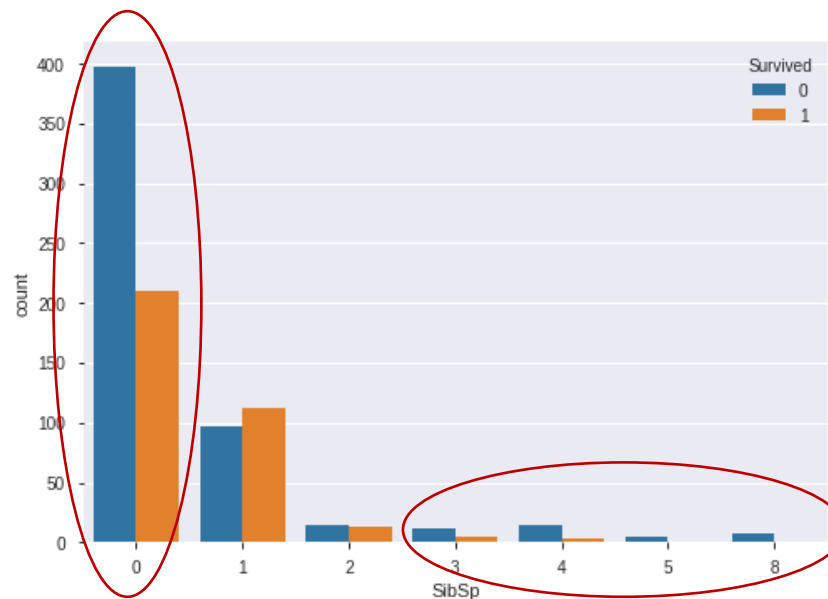


9. 仮説に基づく新しい特徴量の作成 ③

■ SibSp（兄弟姉妹と配偶者の人数）と目的変数との関係

- 散布図や棒グラフが描画できるSeabornパッケージをインポートする.
- SibSpと目的変数の関係を示す棒グラフを作成する.
- SibSpが0のデータと3以上のデータは生存率が低い（死亡した人の方が多い）ことがわかる.

```
import seaborn as sns
sns.countplot(x='SibSp', hue='Survived', data=train)
plt.legend(loc='upper right', title='Survived')
```

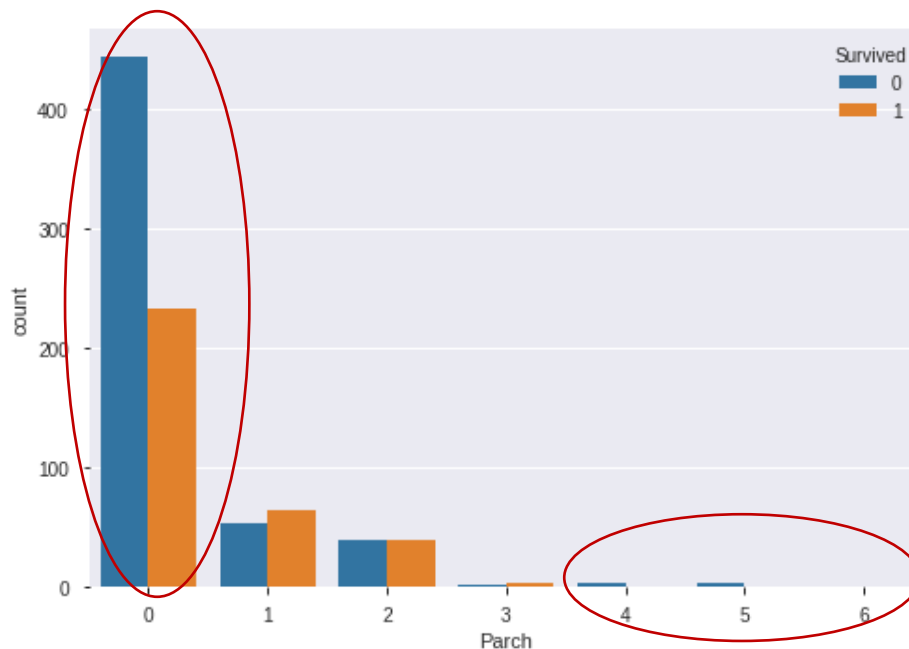


9. 仮説に基づく新しい特徴量の作成 ④

■ Parch (同乗した親と子どもの人数) と目的変数との関係

- Parchと目的変数の関係を示す棒グラフを作成いことがわかる.
- ParchとSibSpについて考えると, どちらも同乗する家族の人数に関する特徴量である.
- 人数が増えると生存率が下がるという共通点もあることから, 「両者を足し合わせて『家族の人数』という特徴量を新たに作ると, 予測性能に寄与する可能性がある」という仮説を立てられる.

```
sns.countplot(x='Parch', hue='Survived', data=train)  
plt.legend(loc='upper right', title='Survived')
```

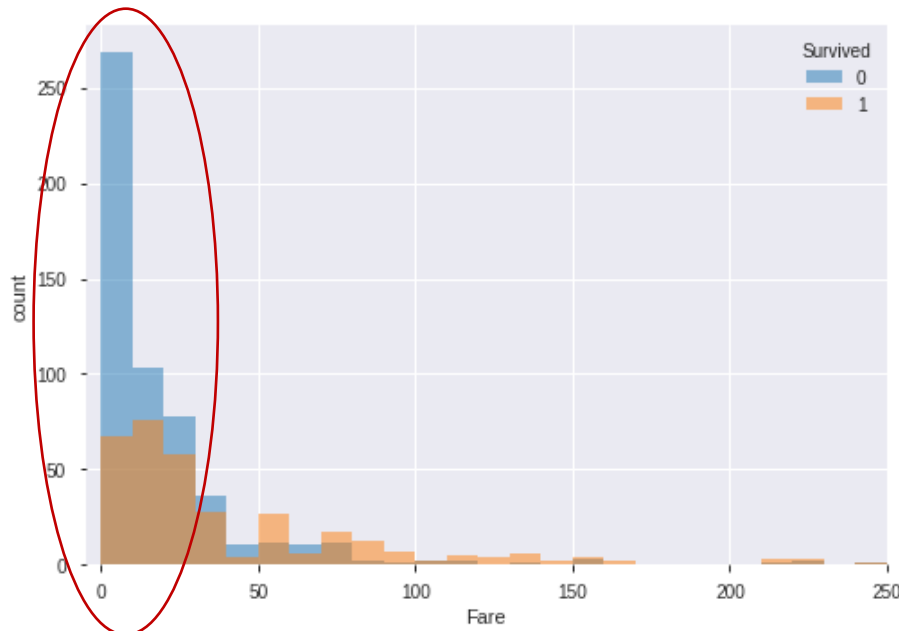


9. 仮説に基づく新しい特徴量の作成 ⑤

■ Fare（運賃）と目的変数との関係

- 目的変数ごとに運賃のヒストグラムをプロットする。
- 運賃が30以下の乗客は生存率が低く、その中でも特に10以下の乗客は生存率が低くなる。

```
plt.hist(train.loc[train['Survived']==0, 'Fare'].dropna(), range=(0,250), bins=25, alpha=0.5, label='0')
plt.hist(train.loc[train['Survived']==1, 'Fare'].dropna(), range=(0,250), bins=25, alpha=0.5, label='1')
plt.xlabel('Fare')
plt.ylabel('count')
plt.legend(title='Survived')
plt.xlim(-5,250)
```

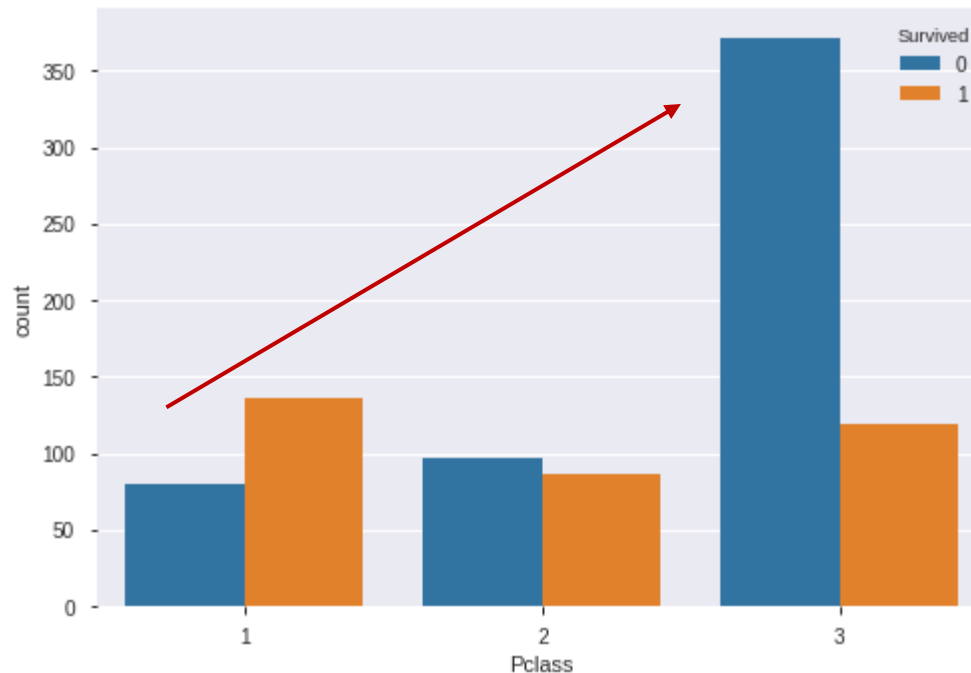


9. 仮説に基づく新しい特徴量の作成 ⑥

■ Pclass (チケットのクラス) と目的変数との関係

- チケットクラスごとに目的変数を集計する.
- Pclassが1から3に増えるにつれ, 生存率が下がるのが分かる.

```
sns.countplot(x='Pclass', hue='Survived', data=train)
```

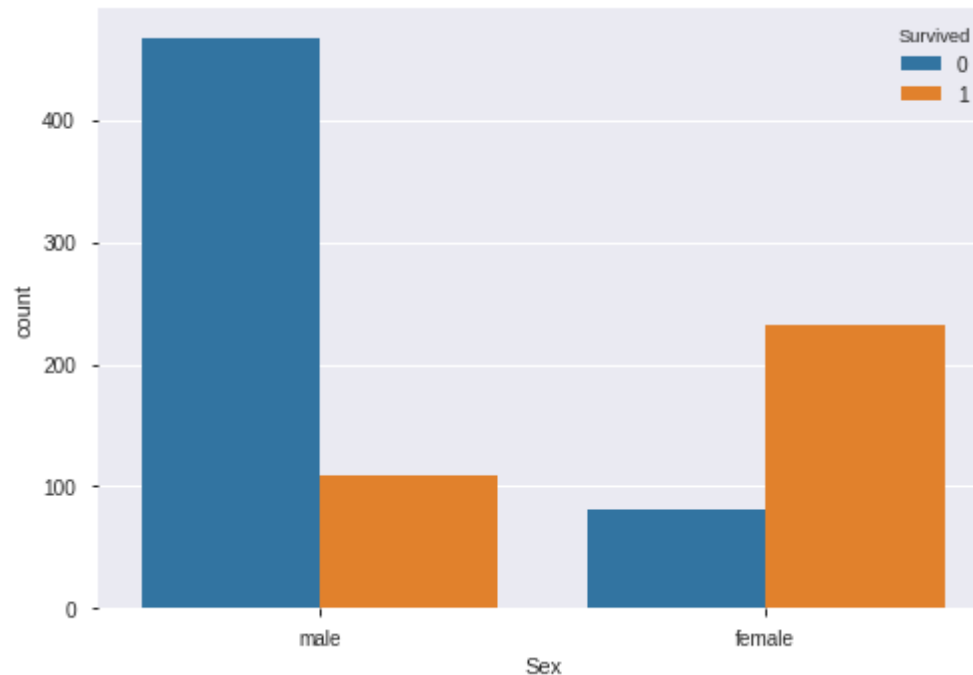


9. 仮説に基づく新しい特徴量の作成 ⑦

■ Sexと目的変数との関係

- 性別ごとに目的変数を集計する
- 男性は生存率が低く、女性は生存率が高く、性別によって生存率に大きな差がある（重要な特徴量）

```
sns.countplot(x='Sex', hue='Survived', data=train)
```

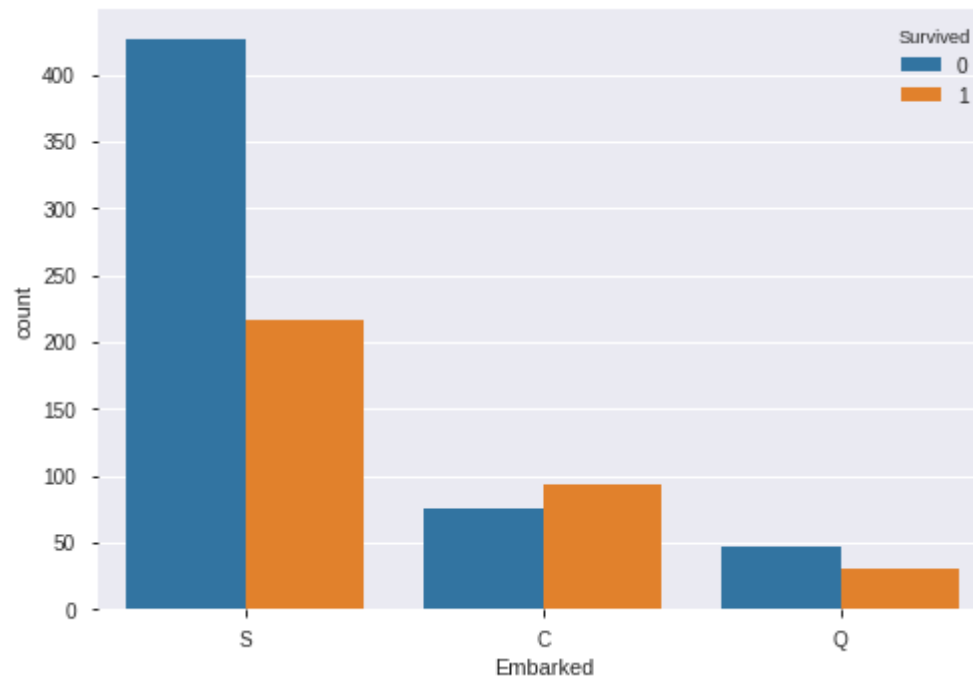


9. 仮説に基づく新しい特徴量の作成 ⑧

■ Embarked（乗船した港）と目的変数との関係

- Embarkedごとに目的変数を集計する
- S とQは生存率が低く、Cは生存率が高いことが分かる

```
sns.countplot(x='Embarked', hue='Survived', data=train)
```



9. 仮説に基づく新しい特徴量の作成 ⑧

Kaggle Notebook: [tuis-study-kaggle-03](#)

■ 再現性の大切さ

- 「**再現性がある**」とは、何度実行しても同じ結果（同じスコア）が得られること。
- 再現性がないと、予測モデルが改善されたか否かを正しく判断できなくなる
- 例えば、特徴量Ageの欠損値を埋める際に**乱数**を使用した場合、平均と標準偏差を考慮した乱数で欠損値を補完したとしても、以下のソースコードでは乱数は実行ごとに値が変わるようになっている。

```
import numpy as np
import pandas as pd

train = pd.read_csv('../input/titanic/train.csv')
test = pd.read_csv('../input/titanic/test.csv')
data = pd.concat([train, test], sort=False)

age_avg = data['Age'].mean()
age_std = data['Age'].std()
np.random.randint(age_avg - age_std, age_avg + age_std)
```

9. 仮説に基づく新しい特徴量の作成 ⑨

■ 再現性の大切さ（続き）

- 再現性を担保するには、次のような方法が考えられる。
 - そもそも乱数を用いる部分を削除する。
 - 乱数のseedを与えて実行結果を固定する。
- Ageについては、そもそも乱数を用いるよりも、欠損していないデータの中央値を与えた方が筋の良い補完ができる可能性がある。以下、中央値で補完するようにソースコードを改変する。

```
data['Age'].fillna(data['Age'].median(), inplace=True)
```

- 機械学習アルゴリズムで乱数を使用する場合、再現性を担保するためにseedを固定しておくのが望ましい。
- seedとは、乱数を作るうえで基点となる値である。以下、ロジスティック回帰のハイパーパラメータとして「random_state=0」を与え、seedを固定している。

```
from sklearn.linear_model import LogisticRegression  
  
clf = LogisticRegression(penalty='l2', solver='sag', random_state=0)
```

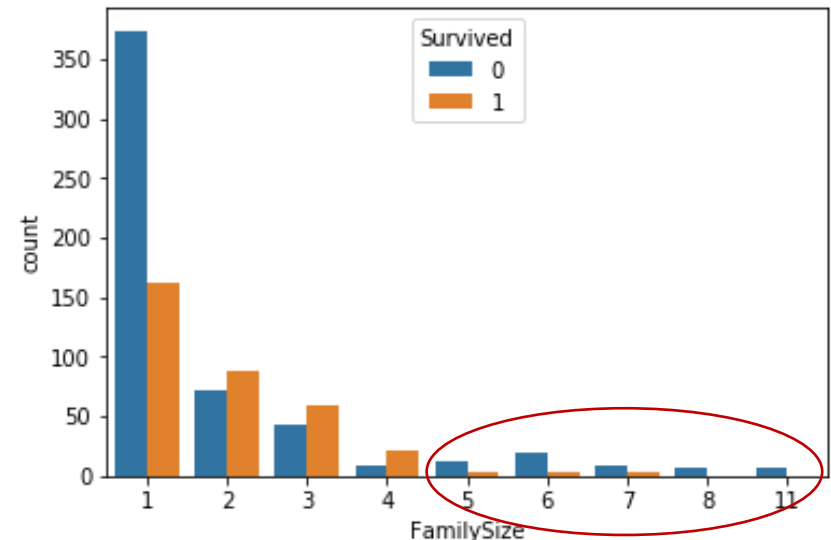
9. 仮説に基づく新しい特徴量の作成 ⑩

■ 仮説に基づいて新しい特徴量をつくる

- 探索的なデータ分析の結果, 「ParchとSibSpの両者を足し合わせて『家族の人数』という特徴量を新たに作ると, 予測性能に寄与する可能性がある」という仮説を立てることができた.
- この仮説の検証のための可視化を行う.
- 新しい列「FamilySize」を作り, その大きさごとに生存したか否かを棒グラフにする.

```
import seaborn as sns
data['FamilySize'] = data['Parch'] + data['SibSp'] + 1
train['FamilySize'] = data['FamilySize'][:len(train)]
test['FamilySize'] = data['FamilySize'][len(train):]
sns.countplot(x='FamilySize', data=train, hue='Survived')
```

- FamilySize ≥ 5 の場合, 死亡が生存を上回っており, 生存率が低いことが分かる.
- 可視化前の仮説が, 確度の高い仮説に変わった.



9. 仮説に基づく新しい特徴量の作成

■ 仮説に基づいて新しい特徴量をつくる

- FamilySize = 1 の人が圧倒的に多く、かつ生存率が低いことが分かる.
- この特徴量も予測性能に寄与しそうなので、以下のようにあたらしく「IsAlone」という特徴量を作成する.

```
data['IsAlone'] = 0
data.loc[data['FamilySize'] == 1, 'IsAlone'] = 1

train['IsAlone'] = data['IsAlone'][:len(train)]
test['IsAlone'] = data['IsAlone'][len(train):]
```

- 作成した特徴量が「有用」だったかを判断するには、次の4つの方法で、学習した結果をsubmitする方法がある.
 - 「FamilySize」と「IsAlone」を加えた場合
 - 「FamilySize」のみを加えた場合
 - 「IsAlone」のみを加えた場合
 - 「FamilySize」と「IsAlone」を加えていない場合

10. いろいろな機械学習アルゴリズム①

Kaggle Notebook: [tuis-study-kaggle-04](#)

- Kaggleのコンペで上位陣での採用が高い「**勾配ブースティング**」について学ぶ.
- **sklearn**を使用. 多くの種類の機械学習が収録されている.
- sklearnでは, clfで宣言するモデルを切り替えるだけで機械学習アルゴリズムを差し替えできる.
- 例えば, 「ランダムフォレスト」の場合は, 以下のようなソースコードになる

```
from sklearn.ensemble import RandomForestClassifier  
clf = RandomForestClassifier(n_estimators=100, max_depth=2, random_state=0)
```

■ LightGBM

- 以下の事前準備が必要である.
 - **学習用・検証用にデータセットを分割する**
 - **カテゴリ変数にリスト形式で宣言する**
- LightGBMは「**決定木**」をもとにした機械学習アルゴリズムである.
- 決定木は, とても単純な機械学習アルゴリズムである. 1つの特徴量に対し1つの閾値を定め, 次々と条件分岐していきながら予測値を決定する.
- 学習データセットから, どの特徴量で, どのような閾値で, どのように判定するかを学ぶ.

10. いろいろな機械学習アルゴリズム②

- LightGMBは**勾配ブースティング**と呼ばれる方法。決定木を大量に作成しながら学習を進める。
- ある時点で作成した決定木の予測結果を確認し、誤差の大きいデータをうまく予測できるように、次の決定木を作成していく。
- 最終的な予測値は、学習の過程で作成したすべての決定木での予測値を利用して算出される。
- 次々と学習をかさねていくため、高い予測性能が期待できる。
- 一方で、学習に利用したデータセットにのみ過剰に適合してしまい、本来の目的である未知の値に対する性能が劣化してしまう「**過学習**」という現象に陥る可能性がある。
- そこで、学習に利用しない検証用データセット（X_valid）に対する性能を見ながら学習を打ち切る「**early stopping**」を利用するのが一般的である。
- X_trainをX_train（学習用）とX_valid（検証用）に分割する。

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_valid, y_train, y_valid = train_test_split(X_train, y_train, test_size=0.3, random_state=0, stratify=y_train)
```

10. いろいろな機械学習アルゴリズム③

■ カテゴリ変数をリスト形式で宣言する

- LightGBMでは、カテゴリ変数に対して特別な処理を自動的に実行してくる。
- 次のように、何をカテゴリ変数として扱ってほしいか明示的にLightGBMに示す。

```
categorical_features = ['Embarked', 'Pclass', 'Sex']
```

- LightGBMの学習・予測を実行する
 - **num_boost_round**（学習の実行回数の最大値）を1000に設定する
 - **early_stopping_rounds**（early_stoppingの判定基準）を10と設定する。連続して10回学習を重ねても検証用データセットに対する性能が改善しなかった場合、学習を打ち切る。

```
import lightgbm as lgb
lgb_train = lgb.Dataset(X_train, y_train, categorical_feature=categorical_features)
lgb_eval = lgb.Dataset(X_valid, y_valid, reference=lgb_train, categorical_feature=categorical_features)

params = { 'objective': 'binary' }
model = lgb.train(params,
                  lgb_train, valid_sets=[lgb_train, lgb_eval], verbose_eval=10,
                  num_boost_round=1000,
                  early_stopping_rounds=10)

y_pred = model.predict(X_test, num_iteration=model.best_iteration)
```

10. いろいろな機械学習アルゴリズム③

■ カテゴリ変数をリスト形式で宣言する（続き）

- 38回目の学習でスコアが最大。そこから（39回目以降から）10回連続でスコアが改善せずに、48回目で学習が打ち切りとなった。

Training until validation scores don't improve for 10 rounds

[10] training's binary_logloss: 0.425241 valid_1's binary_logloss: 0.478975

[20] training's binary_logloss: 0.344972 valid_1's binary_logloss: 0.444039

[30] training's binary_logloss: 0.301357 valid_1's binary_logloss: 0.436304

[40] training's binary_logloss: 0.265535 valid_1's binary_logloss: 0.438139

Early stopping, best iteration is:

[38] training's binary_logloss: 0.271328 valid_1's binary_logloss: 0.435633

- 今回のLightGBMの設定では、出力結果は0~1の連続値になる。
- 今回の閾値は決め打って0.5を上回っていれば1と予測したとしてsubmitする。

```
y_pred = (y_pred > 0.5).astype(int)
y_pred[:10]
```

- ロジスティクス回帰やランダムフォレストのスコアよりも向上しているのが分かる。

■ その他の機械学習アルゴリズム

- XGBoost, CatBoostなどがある。
- PyTorch, TensorFlowなどのパッケージを用いてニューラルネットワークを実装する場合もある。

11. ハイパーパラメータの調整 ①

Kaggle Notebook: [tuis-study-kaggle-05](#)

■ ハイパーパラメータを調節する

- 機械学習アルゴリズムの振る舞いは**ハイパーパラメータ**という値で制御されている。
- ハイパーパラメータの調整は、主に2種類の方法がある。
 - **手動で調整 / チューニングツールを使う (Optuna)**
- データサイズが大きい場合、上記のツールでの調整が現実的な時間で終わらないという問題がある。
- ハイパーパラメータ調整でのスコアの上がり幅は、一般的に、特徴量エンジニアリングで良い特徴量を見つけた場合に劣るので、あまり時間をかけずに手動で微調整する場合も多い。

■ 手動で調整

- これまでLightGBMのparamsには「objective」のみ指定していた。
- ここでは性能を高めるために重要な以下のパラメータを指定する。
 - **max_bin** ... LightGBMは学習を高速化するため各特徴量をいくつかのヒストグラムに変換している。**各特徴量の最大の分割数**を意味し、大きめの値を設定することで、機械学習アルゴリズムの表現力が上がる可能性がある。default値は255なので、300に変更する
 - **learning_rate** ... **学習率**。小さめの値を設定することで「丁寧に」対応関係を学習するようになり、精度向上につながる。default値は0.1なので、0.05に変更する
 - **num_leaves** ... **1つの決定木における分岐の末端の最大数**。大きめの値を設定することで、機械学習アルゴリズムの表現力が上がる可能性がある。default値は31なので、40に変更する

11. ハイパーパラメータの調整 ②

■ 手動で調整（続き）

- 表現力を高める弊害として、**計算量が多くなることや、過学習に陥る可能性が増加する点には注意する必要がある。**
- ここでは3つのハイパーパラメータを一度に変更していますが、それぞれの変更の影響を確認するために、1つずつ調整すると良い。

```
params = {'objective': 'binary', 'max_bin': 300, 'learning_rate': 0.05, 'num_leaves': 40}
```

Training until validation scores don't improve for 10 rounds

[10]	training's binary_logloss: 0.505699	valid_1's binary_logloss: 0.532106
[20]	training's binary_logloss: 0.427825	valid_1's binary_logloss: 0.482279
[30]	training's binary_logloss: 0.377242	valid_1's binary_logloss: 0.456641
[40]	training's binary_logloss: 0.345424	valid_1's binary_logloss: 0.447083
[50]	training's binary_logloss: 0.323113	valid_1's binary_logloss: 0.440407
[60]	training's binary_logloss: 0.302727	valid_1's binary_logloss: 0.434527
[70]	training's binary_logloss: 0.285597	valid_1's binary_logloss: 0.434932

Early stopping, best iteration is:

[66]	training's binary_logloss: 0.293072	valid_1's binary_logloss: 0.433251
------	-------------------------------------	---

変更前よりも小さな値になっている。
この値は損失なので、小さい方が望ましい。

11. ハイパーパラメータの調整 ③

■ Optunaを使う

```
import optuna
```

- 手動でハイパーパラメータを調整していると,
 - 「大きめ」「小さめ」といっても、具体的にどの値にすれば良いのかわからない
 - 各パラメータの組み合わせ方もいくつかあり、逐一設定・実行して性能を検証するのは大変
- このような課題を解決してくれるのが、ハイパーパラメータのチューニングツールである。
 - Grid search
 - Bayesian Optimization
 - Hyperopt
 - **Optuna** など
- Optunaを使うには、あらかじめ次の関数内の**trial.suggest_int()**のように、探索範囲を定義する。

```
params = {  
    'objective': 'binary',  
    'max_bin': trial.suggest_int('max_bin', 255, 500),  
    'learning_rate': 0.05,  
    'num_leaves': trial.suggest_int('num_leaves', 32, 128),  
}
```

11. ハイパーパラメータの調整 ④

■ Optunaを使う（続き）

- learning_rateの調整はしていない。テーブルデータをLightGBMで扱う場合、一般にlearning_rateが低いほど高い性能が得られるため、そのため探索範囲には含め得ず、必要であれば後に手動で低い値に変更する。
- n_trialsは試行回数である。ここでは計算を短くするため、40回程度にしておく。
- 乱数も固定する。

```
study = optuna.create_study(sampler=optuna.samplers.RandomSampler(seed=0))  
study.optimize(objective, n_trials=40)
```

- 指定した範囲内で試行回数だけ探索した結果得られた最良のハイパーパラメータが表示される。

```
{'max_bin': 427, 'num_leaves': 79}
```

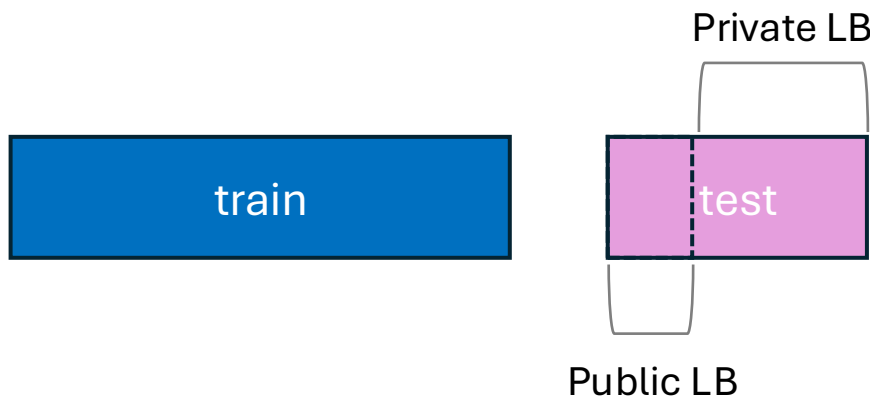
- 手動で調整するにせよ、チューニングツールを使うにせよ、機械学習アルゴリズムをブラックボックス的に利用するのではなく、ハイパーパラメータを正しく理解することが非常に大切である。

12. スコアの検証 ①

Kaggle Notebook: [tuis-study-kaggle-06](#)

■ 「Cross Validation」の大切さ

- 機械学習モデルの性能を見積もる「**validation**」について学ぶ
- ここまでkaggleにsubmitした際のスコアで確認していたが、この方法には次のような問題がある。
 - **1日のsubmit回数に制限がある**
 - スコアが上がる保証もないのに気軽にsubmitするのは得策ではない
 - 1日のsubmit回数分しか試行錯誤ができない状況にも陥ってしまう
 - **Public LBで良いスコアが出ても、一部のデータのみで過学習した結果の可能性がある**
 - 最終順位は残りのPrivate LBのデータに対する性能で決定する。
 - Public LBに過学習している可能性もあり、Private LBでの性能に寄与するかは分からない
 - Public LBとPrivate LBの分割方法は参加者に開示されない場合が多い



12. スコアの検証 ②

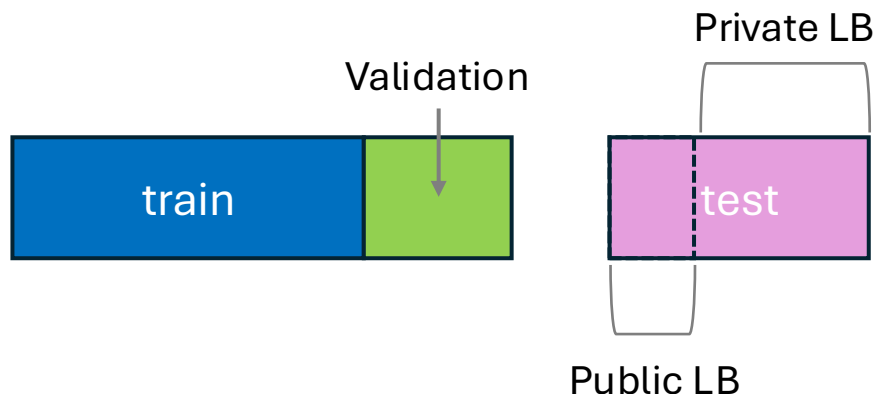
■ 学習用データセットから検証用データセットを作る

- 学習用データセットから**検証用データセット**を作成し、自分のモデルの性能を測るのが一般的.
- submitしなくても手元でスコアを計算できるので, submit回数に制限があるという問題に対処できる.
- 検証用データセットの作成方法次第ではあるが、全体像の見ていないPublic LBのスコアに比べて、信頼に足りる存在になる可能性がある.

■ ホールドアウト検証

- LightGBMを利用する段階で「**ホールドアウト検証**」と呼ばれる一種のvalidationを実施している.
- 学習用データセットを分割したうえで, LightGBMを学習させている

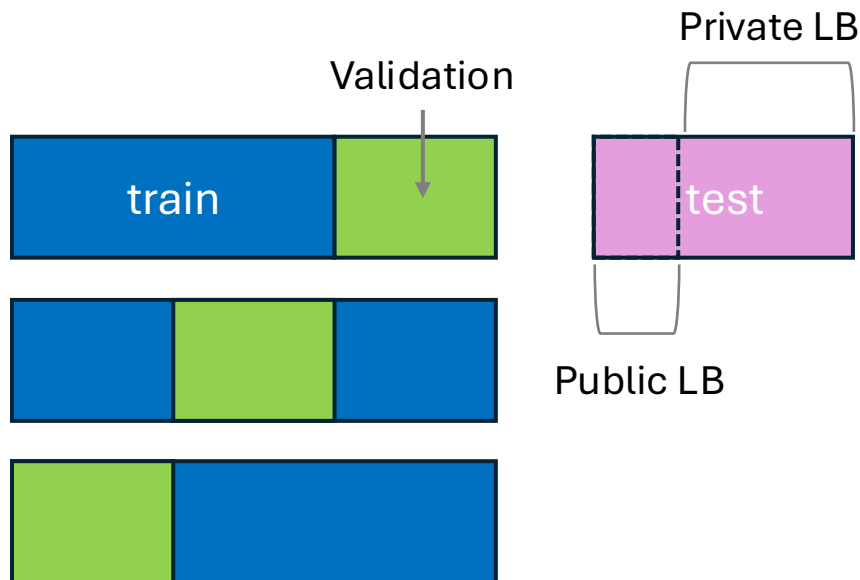
```
X_train, X_valid, y_train, y_valid  
= train_test_split(X_train, y_train, test_size=0.3, random_state=0, stratify=y_train)
```



12. スコアの検証 ③

■ Cross Validation (CV, 交差検証)

- 複数回に渡って異なる方法でデータセットを分割し、それぞれでホールドアウト検証を実行する方法
- それぞれのスコアの平均を確認することで、1回のホールドアウト検証で生じる偏りに対する懸念を弱める
- `train_test_split()`を複数回用いて実装することも可能だが、sklearnのKFoldというクラスを利用できる。
- KFoldの`n_splits`は分割数である。



```
from sklearn.model_selection import Kfold  
  
cv = KFold(n_splits=5, shuffle=True, random_state=0)
```

12. スコアの検証 ④

■ Cross Validation (CV, 交差検証) (続き)

- CVを実施したときは、**各分割でのスコアの平均**をスコア (CVスコア) と見なすことが多い.
- 学習用データセットを分割した最小単位をそれぞれ「**fold**」と呼び、各分割で学習に使われなかったfoldは「**Out-of-fold (oof)**」と表現される. oof_train という変数名は「trainのoff」という意味.
- 各分割でのoofに対する予測値を格納している.

```
pd.DataFrame(oof_train).to_csv('oof_train_kfold.csv', index=False)

scores = [
    m.best_score['valid_1']['binary_logloss'] for m in models
]
score = sum(scores) / len(scores)
print('===CV scores===')
print(scores)
print(score)
```

- ホールドアウト検証では検証用データセットに該当する部分を学習に利用できていないですが、Cross Validationでは複数の分割を実施するので、全体として与えられたデータセットを漏れなく学習に利用できている.

12. スコアの検証 ⑤

■ データセットの分割方法

- データセットの分割にあたっては、データセットや課題設定の特徴を意識するのが大切。
- KFoldは、特にデータセットや課題設定の特徴を考慮することなくデータセットを分割する。
- 例えば、学習用・検証用データセット内の「目的変数が1 ($y==1$)」の割合を見てみると、次のように fold_idが2と4の場合などで、顕著に割合が異なっている。

```
from sklearn.model_selection import KFold

cv = KFold(n_splits=5, shuffle=True, random_state=0)
for fold_id, (train_index, valid_index) in
    enumerate(cv.split(X_train)):
    X_tr = X_train.loc[train_index, :]
    X_val = X_train.loc[valid_index, :]
    y_tr = y_train[train_index]
    y_val = y_train[valid_index]

    print(f'fold_id: {fold_id}')
    print(f'y_tr y==1 rate: {sum(y_tr)/len(y_tr)}')
    print(f'y_val y==1 rate: {sum(y_val)/len(y_val)}')
```

```
old_id: 0
y_tr y==1 rate: 0.38342696629213485
y_val y==1 rate: 0.3854748603351955
fold_id: 1
y_tr y==1 rate: 0.3856942496493689
y_val y==1 rate: 0.37640449438202245
fold_id: 2
y_tr y==1 rate: 0.39831697054698456
y_val y==1 rate: 0.3258426966292135
fold_id: 3
y_tr y==1 rate: 0.3856942496493689
y_val y==1 rate: 0.37640449438202245
fold_id: 4
y_tr y==1 rate: 0.36605890603085556
y_val y==1 rate: 0.4550561797752809
```

12. スコアの検証 ⑥

■ データセットの分割方法（続き）

- 割合を保ったままCVを実施するためには、sklearnの **StratifiedKFold()** が利用可能である。
- 学習用・検証用データセット内の「y==1」の割合が可能な範囲で均一に保たれる

```
from sklearn.model_selection import StratifiedKFold

cv = StratifiedKFold
(n_splits=5, shuffle=True, random_state=0)
for fold_id, (train_index, valid_index) in
    enumerate(cv.split(X_train)):
    X_tr = X_train.loc[train_index, :]
    X_val = X_train.loc[valid_index, :]
    y_tr = y_train[train_index]
    y_val = y_train[valid_index]

    print(f'fold_id: {fold_id}')
    print(f'y_tr y==1 rate: {sum(y_tr)/len(y_tr)}')
    print(f'y_val y==1 rate: {sum(y_val)/len(y_val)}')
```

```
old_id: 0
y_tr y==1 rate: 0.38342696629213485
y_val y==1 rate: 0.3854748603351955
fold_id: 1
y_tr y==1 rate: 0.38342696629213485
y_val y==1 rate: 0.3854748603351955
fold_id: 2
y_tr y==1 rate: 0.38429172510518933
y_val y==1 rate: 0.38202247191011235
fold_id: 3
y_tr y==1 rate: 0.38429172510518933
y_val y==1 rate: 0.38202247191011235
fold_id: 4
y_tr y==1 rate: 0.38375350140056025
y_val y==1 rate: 0.384180790960452
```

12. スコアの検証 ⑦

■ その他，分割の際に気をつけたいこと

- データセット内に時系列性がないか
- データセット内にグループが存在しないか

■ データセット内に時系列性がないか

- 学習用データセットから1週間を開けたPrivate LBに対する性能を高めるのが目的になるコンペの場合，検証用データセットを作成する際も，一定期間を空けてデータセットを分割するのが理想的である．
- 状況に合わせて，自分でデータセットの分割を定義するのが現実的である．

■ データセット内にグループが存在しないか

- データセット内にグループが存在する場合，同一グループ内の予測が比較的容易である点に注意する
- 例えば，ドライバーの画像（1人のドライバーに対して複数の画像）から運転態度を10クラスに分割する問題．学習用データセットと，テスト用データセットの間にドライバーの重複がない点に注意する．
- 同一のドライバーの画像を予測するのは比較的簡単なので，テスト用データセットに対する性能が不当に高く見積もられてしまう．

13. アンサンブル学習 ①

Kaggle Notebook: [tuis-study-kaggle-07](#)

- 複数の機械学習モデルを組み合わせることで性能の高い予測値を獲得する手法.
- 10個のyに対して, 0から1の二値分類を考える, 次のような予測をする問題である.

$$(y_0, y_1, \dots, y_9) = (0, 1, \dots, 0)$$

- 以後, 右辺の括弧内の数字だけを取り出して次のように表現する.
- 例えば, 「000010001」は, 「y₄とy₉のみを1と予測した」という意味.
- 正解がすべて「111111111」であるような問題を考える.

- この問題に対して, モデルA・B・Cがそれぞれ次のような予測をしたとする

- モデルA = 正解率80%

111111100

- モデルB = 正解率70%

0111011101

- モデルC = 正解率60%

1000101111

- 単純な「**多数決**」によるアンサンブルの場合, 以下のような結果になる

1111111101 → 正解率 = 90%

- **それぞれのモデルの良い部分・悪い部分を補完し合い, 全体として良い予測結果が得れる.**

13. アンサンブル学習 ②

■ Titanicで実験

- これまで作成してきたcsvファイルを用いて、実際にアンサンブル効果を確認する.
- ランダムフォレストとLightGBMによる3つのcsvファイルを利用する
 - submission_lightgbm_skfold.csv ... StratifiedKFOLD()を用いて作成したcsv
 - submission_lightgbm_holdout.csv ... ホールドアウト検証を用いて作成したcsv
 - submission_randomforest.csv ... ランダムフォレストを用いて作成したcsv

```
sub_lgbm_sk = pd.read_csv('../input/submit-files/submission_lightgbm_skfold.csv')
sub_lgbm_ho = pd.read_csv('../input/submit-files/submission_lightgbm_holdout.csv')
sub_rf = pd.read_csv('../input/submit-files/submission_randomforest.csv')
```

- 最初に予測がどれくらい類似しているのかを確認するため、それぞれの相関を計算する.

	sub_lgbm_sk	sub_lgbm_ho	sub_rf
sub_lgbm_sk	1.000	0.883	0.796
sub_lgbm_ho	0.883	1.000	0.731
sub_rf	0.796	0.731	1.000

- アンサンブルの観点では多様性が大切なので、**予測値の相関が小さい方が望ましい**.
- ただし、相関の大きさは絶対的な基準はない. アンサンブルの文脈では0.95以下ならば十分に相関が小さいと言える.

13. アンサンブル学習 ②

■ Titanicで実験（続き）

- これまでここでは多数決で予測値を決定する.
- 3ファイルの予測値部分を合計し, 合計が2以上の場合は全体として予測値を1とする.

```
ub = pd.DataFrame(pd.read_csv('../input/titanic/test.csv')['PassengerId'])
sub['Survived'] = sub_lgbm_sk['Survived'] + sub_lgbm_ho['Survived'] + sub_rf['Survived']
sub['Survived'] = (sub['Survived'] >= 2).astype(int)
sub.to_csv('submission_lightgbm_ensemble.csv', index=False)
```

- submiとしてスコアを確認してください.

本講義のまとめ

Kaggleに参加するため必要な準備をする

1. Kaggleとは
2. Kaggleで用いる機械学習 (KW: 教師あり学習)
3. Kaggleのアカウントの作成
4. Competitionsページの概要
5. 環境構築不要なNotebooksの使い方

最低限必要な機械学習テクニックを身につける

6. Kaggleへのsubmitの方法 (KW: Leaderboard, 特徴量エンジニアリング)
7. 一連の処理の流れ
8. 探索的データ分析 (KW: Pandas Profiling, 説明変数, 目的変数)
9. 仮説に基づく新しい特徴量の作成 (KW: 再現性)
10. いろいろな機械学習アルゴリズム (KW: sklearn, LightGBM, 勾配ブースティング)
11. ハイパーパラメータの調整 (KW: Optuna)
12. スコアの検証 (KW: ホールドアウト検証, Cross Validation)
13. アンアンブル学習 (KW: 多数決)